

本科毕业设计说明书（论文）
Undergraduate Graduation Project Report (Thesis)

面向体感游戏的动态手势轨迹跟踪方法实现
IMPLEMENTATION OF DYNAMIC GESTURE
TRAJECTORY TRACKING METHOD FOR
MOTION-SENSING GAMES

学 院： 计算机科学与技术学院、软件学院

专 业： 软件工程（中外合作办学）

班 级：

学 号：

学生姓名： 曹逸天

指导老师：

提交日期： 2026 年 6 月

面向体感游戏的动态手势轨迹跟踪方法实现

摘要

随着计算机视觉和实时交互技术的发展，基于普通 RGB 摄像头的体感交互逐渐成为低成本人机交互的重要方向。相比键盘、鼠标和手柄等传统输入方式，手势输入能够以更加自然的方式表达玩家意图，适合用于体感游戏中的移动、确认、施法和防御等交互行为。然而，动态手势并不是单帧图像分类问题，而是包含位移、

方向、速度、持续时间和状态变化的连续轨迹识别问题。如何将视觉识别结果稳定转换为游戏系统可以消费的交互命令，是体感游戏工程实现中的关键问题。

本文面向普通 RGB 摄像头下的体感游戏场景，设计并实现了一套动态手势轨迹跟踪与 Unity 游戏交互系统。系统以 MediaPipe 手部关键点识别和外部视觉桥接输入为基础，构建 Mock、Native MediaPipe 和 ExternalBridge 三种输入源，并将不同来源的识别结果统一抽象为手势帧、手势命令、命令历史和动态动作事件。系统通过基于时间窗、位移阈值、状态变化和冷却机制的规则方法识别滑动、响指、指向到握拳等动态手势，并将其映射到 Unity 游戏原型《符印守卫》中的菜单导航、玩家移动、法术释放、护盾防御和战斗反馈。

在系统实现方面，本文完成了 Unity 原型场景、输入路由、手势识别、玩家控制、敌人生成、HUD 调试反馈、训练集校验和性能监控等模块，并进一步实现了自定义动态手势模板、在线增强采集和验证界面反馈机制。实验与测试部分通过 EditMode 和 PlayMode 自动化测试验证场景生成、输入适配、动态动作识别、序列匹配和数据集结构检查等功能，并设计了 Mock、Native MediaPipe 和 ExternalBridge 三种模式下的实时性能评价指标。结果表明，该系统能够形成从视觉输入到游戏反馈的完整闭环，为普通摄像头条件下的体感游戏交互提供了一种可复现、可扩展的工程实现方案。同时，自定义手势增强采集实验和基于 Jester 训练集的采样回放验证也表明，少样本模板在真实摄像头交互中仍存在误触发、漏检和样本边界不清等问题，而更大规模的采样挖掘与离线回放可以提升规则模板的稳定性，但仍需要结合负样本、候选样本筛选和自动阈值估计进一步改进。

关键词：动态手势识别；体感游戏；MediaPipe；Unity；轨迹跟踪；人机交互

Abstract

With the development of computer vision and real-time interaction technologies, gesture-based interaction using ordinary RGB cameras has become a feasible low-cost approach for natural human-computer interaction. Compared with traditional input devices such as keyboards, mice and game controllers, gesture input allows players to express intentions in a more natural way and is suitable for movement, confirmation, spell casting and defense actions in motion-sensing games. However, dynamic gesture recognition is not a single-frame image classification problem. It involves continuous trajectory information such as displacement, direction, speed, duration and state transition. Therefore, how to convert visual recognition results into stable and game-consumable commands is a key engineering challenge.

This thesis designs and implements a dynamic gesture trajectory tracking and Unity game interaction system for motion-sensing games under ordinary RGB camera conditions. Based on MediaPipe hand landmark detection and an external vision bridge, the system provides three input sources: Mock, Native MediaPipe and ExternalBridge. Recognition results from different sources are unified into gesture frames, gesture commands, command history and motion gesture events. Rule-based methods using time windows, displacement thresholds, state transitions and cooldown mechanisms are applied to recognize dynamic gestures such as swipes, snaps and point-to-fist transitions. These commands are then mapped to menu navigation, player movement, spell casting, shield defense and combat feedback in the Unity prototype game Spell Guard.

The implementation includes Unity scenes, input routing, gesture recognition, player control, enemy spawning, HUD feedback, dataset validation and performance monitoring. A custom dynamic gesture template mechanism, online sample augmentation workflow and validation interface are also implemented for extensibility analysis. EditMode and PlayMode tests are used to verify scene generation, input adapters, motion recognition, sequence matching and dataset validation. The thesis also proposes runtime performance metrics under Mock, Native MediaPipe and ExternalBridge modes. The results show that the system forms a complete interaction loop from visual input to game feedback, providing a reproducible and extensible engineering solution for camera-based motion-sensing game interaction. The custom gesture augmentation experiment and the Jester-based sampled replay validation further indicate that few-sample templates still suffer from false triggers, missed detections and unclear sample boundaries in real camera interaction, while larger sampled mining sets can improve rule-template stability. These findings motivate future improvements using negative samples, candidate sample filtering and automatic threshold estimation.

Keywords: dynamic gesture recognition; motion-sensing game; MediaPipe; Unity; trajectory tracking; human-computer interaction

第 1 章 绪论

1.1 研究背景

人机交互方式随着计算机硬件、传感器和人工智能技术的发展不断演进。早期电子游戏主要依赖键盘、鼠标、手柄等物理输入设备，这类输入方式具有响应稳定、学习成本低和工程实现成熟等优点，但玩家的操作仍然需要通过按钮、摇杆或屏幕

控件间接表达。对于强调身体参与、动作反馈和沉浸体验的体感游戏而言，传统输入方式难以充分体现玩家动作与游戏世界之间的自然对应关系。

体感游戏通过摄像头、深度传感器、惯性传感器或专用体感设备采集玩家动作，并将其转换为游戏控制指令。早期体感方案通常依赖 Kinect、数据手套或专用外设，这些设备能够提供较稳定的深度信息或关节数据，但也带来了成本、部署环境和设备依赖等问题。相比之下，普通 RGB 摄像头具有价格低、获取方便、部署简单的优势，更适合作为毕业设计原型、轻量化交互系统和大众化应用场景中的输入设备。手势识别在人机交互中的应用价值已在多篇综述中得到讨论，相关研究也表明视觉手势适合用于低成本、自然化的交互场景[8-12]。

近年来，MediaPipe、YOLO 等计算机视觉框架的发展降低了普通摄像头下人体姿态、手部关键点和目标检测的实现门槛。MediaPipe Hands 能够输出手部 21 个关键点，为手势姿态分析和动态轨迹建模提供基础数据[1-4,19]；YOLO 系列目标检测算法能够在复杂背景中定位目标区域，为后续关键点提取提供更稳定的输入范围[5-7]。将这类视觉算法与 Unity 等游戏引擎结合，可以构建低成本、实时、可交互的体感游戏系统[20]。

但是，体感游戏中的手势交互并不只是识别某一帧图像中的手势类别。以施法、移动、菜单切换和确认操作为例，许多交互意图都依赖连续动作轨迹，例如向左滑动、向右滑动、从指向变为握拳、拇指和中指靠近后分离等。这类动态手势包含时间、速度、方向和状态变化，需要在连续帧中进行跟踪和判断。因此，本文将研究重点放在“动态手势轨迹跟踪与命令映射”上，而不是单纯的静态手势分类。

1.2 研究问题

普通 RGB 摄像头下的动态手势体感游戏需要同时解决视觉识别、动作建模和游戏交互三个层面的问题。

第一，普通摄像头缺少深度信息，手部检测容易受到光照变化、背景复杂度、摄像头角度、遮挡和运动模糊的影响。当手部离开画面、快速移动或与背景颜色接近时，识别结果可能出现短暂丢失或抖动。

第二，动态手势具有明显的个体差异。不同玩家完成同一动作时，动作幅度、速度、轨迹形状和持续时间可能不同。如果仅依赖某一帧的手势类别，难以准确表达滑动、挥动、切换、确认等连续交互意图。

第三，视觉识别结果不能直接驱动游戏逻辑。游戏系统需要的是稳定、明确、可解释的语义命令，例如“向左移动”“释放火焰术”“确认当前菜单项”等。因此，视觉层输出的关键点、置信度和手势标签需要经过抽象、过滤和映射，才能被玩法、UI 和战斗模块消费。

第四，游戏交互对实时性和反馈连续性要求较高。玩家做出动作后，如果系统反馈延迟明显，或者因为单帧误识别频繁触发错误命令，就会影响游戏体验。因此，系统不仅需要关注识别功能是否可用，还需要关注帧率、延迟、误触、触发稳定性和演示容错能力。

1.3 研究目标

本文的研究目标是设计并实现一套面向体感游戏的动态手势轨迹跟踪与 Unity 交互系统，使普通摄像头或外部视觉输入能够转化为稳定的游戏控制命令，并在实际游戏原型中完成验证。具体目标包括：

（1）设计普通 RGB 摄像头下的手势输入与关键点获取方案，支持 Unity 内部 Native MediaPipe 输入和外部视觉桥接输入。

（2）构建统一的手势运行时抽象，将不同输入源的数据转换为手势帧、手势命令、命令历史和动态动作事件。

（3）设计基于连续帧轨迹、时间窗、状态变化和阈值判断的动态手势识别方法，实现滑动、响指、指向到握拳等动作识别。

（4）在 Unity 中实现体感施法游戏原型《符印守卫》，完成菜单、训练、移动、施法、敌人、生命、护盾和反馈等功能。

（5）通过自动化测试、运行截图、性能指标和不同输入模式对比，验证系统的可用性、实时性和工程可复现性。

1.4 主要工作

本文围绕动态手势轨迹跟踪和游戏交互闭环完成了以下工作：

（1）构建多输入源统一框架。系统支持 Mock、Native MediaPipe 和 ExternalBridge 三种输入模式。Mock 模式用于无摄像头环境下的开发、测试和答辩兜底；Native MediaPipe 模式用于 Unity 内部摄像头与手部关键点识别；ExternalBridge 模式用于接收外部 Python 视觉链路或离线回放数据。

（2）设计手势命令抽象。系统将底层识别结果封装为 GestureFrame、GestureCommand、GestureCommandHistory 和动态动作事件，使玩法层不直接依赖某一种视觉识别实现。这种设计降低了输入层与游戏逻辑之间的耦合度，也为后续扩展新的手势、输入源或识别算法提供了接口基础。

（3）实现动态手势识别与序列匹配。系统通过关键点轨迹、时间窗口、位移阈值、状态转换和冷却时间识别滑动、响指、指向到握拳等动作，并通过命令历史和序列匹配机制支持未来组合动作扩展。

(4) 完成 Unity 游戏原型集成。本文以《符印守卫》为验证场景，实现了手势驱动的开始菜单、教学训练、第一人称移动、火焰法术、冰霜法术、护盾防御、敌人生成、胜负结算和 HUD 调试反馈。

(5) 建立测试与实验支撑。项目包含 EditMode 和 PlayMode 自动化测试，覆盖场景生成、输入适配、动态识别、序列匹配、训练集校验和性能监控等内容，并设计了 FPS、P95 帧耗时、UDP 包间隔、估算延迟、静态命令数量和动态命令数量等实验指标。

1.5 论文结构

本文共分为七章。第 1 章介绍研究背景、研究问题、研究目标和主要工作。第 2 章介绍体感游戏、视觉手势识别、MediaPipe、YOLO 和 Unity 游戏开发相关技术。第 3 章分析系统需求并给出总体架构设计。第 4 章阐述动态手势轨迹跟踪与命令识别方法。第 5 章介绍 Unity 游戏原型《符印守卫》的系统实现。第 6 章给出测试与实验设计，并分析系统功能、性能和局限性。第 7 章总结全文工作并展望后续改进方向。

第 2 章 相关技术与研究现状

2.1 体感游戏交互技术

体感游戏是指通过玩家身体动作、手势或姿态变化来控制游戏过程的一类交互系统。与传统键鼠或手柄输入相比，体感交互更强调身体参与和动作反馈，能够增强玩家的沉浸感和临场感。体感游戏不仅应用于娱乐领域，也常见于康复训练、体育教学、认知干预和特殊教育等场景[19-22]。

从输入设备来看，体感交互可以分为专用传感器方案和普通摄像头方案。专用传感器如 Kinect、VR 控制器和数据手套能够提供较稳定的深度、惯性或手部姿态数据，但往往需要额外硬件支持。普通 RGB 摄像头方案只需要常见摄像头设备，部署成本低，但需要依赖计算机视觉算法从二维图像中估计手部或身体关键点。

对于游戏系统而言，体感交互不仅要完成动作识别，还要满足实时性和稳定性要求。游戏场景中的每一次输入都可能影响角色移动、技能释放或战斗结果，因此系统需要避免频繁误触，并在识别结果不稳定时提供合理的容错机制。本文选择体感施法防守游戏作为验证场景，正是因为该场景对动作识别、命令映射和实时反馈都有较明确的需求。

2.2 视觉手势识别技术

视觉手势识别方法大致可以分为传统视觉方法、深度学习分类方法、目标检测方法和关键点轨迹方法。传统视觉方法通常使用肤色分割、背景差分、轮廓检测和

手工特征描述手部区域，优点是实现简单、计算量较低，但在光照变化、背景复杂和肤色差异较大的环境下稳定性不足。国内外综述通常将这一路线作为视觉手势识别的早期基础，并进一步比较了深度学习、关键点建模和时序识别方法的适用条件[8-12]。

深度学习方法通过卷积神经网络或目标检测模型自动学习图像特征，能够在复杂背景下获得更强的鲁棒性。YOLO 系列算法是一类典型的单阶段目标检测方法，能够在较高速度下完成目标定位，因此常用于人体、手部或交互对象检测[5-7]。对于手势交互而言，YOLO 更适合作为前置定位模块，而不是直接替代关键点估计模块。它可以先从整幅图像中定位人或手部区域，减少背景干扰，再将候选区域交给后续关键点算法处理。

关键点方法则关注手部骨架或人体骨架的空间结构。MediaPipe Hands 能够从图像中估计 21 个手部关键点，包括手腕、掌指关节和指尖等位置[1-4]。相比直接对整张图像进行分类，关键点方法更适合描述手指弯曲、掌心移动、指尖距离变化等结构化信息，也更便于构建动态轨迹识别规则。近年研究也进一步将 MediaPipe 关键点与 LSTM、GRU、Transformer 等模型结合，用于实时或动态手势识别[15-17,19]。

因此，YOLO 与 MediaPipe 在本文系统中的关系不是并列替代关系，而是“前置检测 + 精细关键点估计”的互补关系。YOLO 关注图像中目标区域的位置，解决的是“从哪里看”的问题；MediaPipe 关注目标区域内手部关键点的几何结构，解决的是“手是什么姿态、关键点如何移动”的问题。对于动态手势轨迹跟踪而言，真正进入 Unity 命令层的是关键点序列、手势标签、置信度和时间戳，而不是 YOLO 检测框本身。

结合当前项目实现，系统主线以 MediaPipe 手部关键点和外部视觉桥接数据为基础。Native MediaPipe 链路在 Unity 内部直接获取摄像头图像并生成手部关键点；ExternalBridge 链路通过 UDP 接收外部程序输出的 ExternalVisionFrame，其中可以包含 handLandmarks、poseLandmarks、gesture、confidence、timestamp 和 source 等字段。如果外部程序启用 YOLO，则 YOLO 可以在桥接程序中作为前置检测步骤参与图像裁剪或人体区域定位，随后仍由 MediaPipe 或其他关键点模块输出 Unity 可消费的结构化帧。因此，本文不声称完成了独立的大规模 YOLO 手势分类模型训练，而是将 YOLO + MediaPipe 作为外部视觉链路的增强方案和后续对比实验方向。

2.3 动态手势轨迹建模方法

静态手势识别主要关注单帧或短时间稳定姿态，例如握拳、张掌、V 字手势等。动态手势识别则需要分析连续帧之间的位置变化、速度变化和状态转换。例如，向

左滑动需要判断掌心或关键点在一段时间内的横向位移是否超过阈值；响指动作需要观察拇指和中指之间距离从接近到分离的变化过程；从指向到握拳则需要识别手势类别的状态转换。

动态轨迹建模方法可以分为基于规则的方法和基于时序模型的方法。基于规则的方法通常使用时间窗、位移阈值、速度阈值、方向约束和冷却时间判断动作是否成立，也可以借鉴 DTW 等经典时间序列匹配思想处理不同长度动作之间的对齐问题[18]。这类方法计算量小、实时性好、可解释性强，适合毕业设计原型和规则明确的游戏交互场景。基于时序模型的方法如 LSTM、GRU 和 Transformer 能够学习更复杂的动作模式，但需要更多标注数据和训练成本[13-17]。

本文当前实现重点采用“规则 + 时间窗 + 命令历史”的工程方案。该方案能够较快接入 Unity 游戏逻辑，并便于解释每类动作的触发条件。对于更复杂的连续手势、个体差异适应和多手组合动作，后续可引入深度时序模型进行改进。

2.4 Unity 游戏开发与测试技术

Unity 是常用的跨平台游戏引擎，采用组件化开发方式，通过 GameObject、Component 和 MonoBehaviour 组织场景对象与逻辑脚本。Unity 的优势在于可视化编辑、运行时调试、跨平台构建和丰富的 UI、物理、动画、音频生态。对于体感游戏原型而言，Unity 能够快速完成三维场景、角色控制、敌人逻辑、UI 反馈和输入系统集成。

本文项目使用 Unity 2022.3.62f2c1，主要依赖包括 MediaPipe Unity 包、Unity Test Framework、UGUI、Timeline 和 Visual Scripting 等[25-26]。其中 Unity Test Framework 支持 EditMode 和 PlayMode 测试，可用于验证编辑器工具、运行时逻辑和场景组件绑定。本文通过测试覆盖场景生成、输入适配、动态识别、序列匹配和训练集校验等模块，提升系统的可复现性和可靠性。

2.5 游戏交互评价指标

手势识别系统的评价不能只关注分类准确率。在体感游戏场景中，玩家更关心输入是否及时、反馈是否连续、误触是否较少，以及系统在不同环境下是否稳定。因此，本文将评价指标分为功能指标和交互性能指标。

功能指标包括菜单是否可操作、手势是否能触发对应法术、敌人是否能被击中、胜负流程是否完整、不同输入模式是否能正常切换等。性能指标包括平均 FPS、P95 帧耗时、UDP 包平均间隔、估算链路延迟、静态命令触发次数、动态命令触发次数和手部检出比例等。若后续补充用户实验，还可以增加任务完成时间、手势触发成功率和误触率等指标。

第3章 系统需求分析与总体设计

3.1 应用场景分析

本文实现的应用场景为 Unity 体感施法游戏原型《符印守卫》。游戏面向毕业设计演示，核心玩法是玩家通过手势完成菜单选择、训练引导、第一人称移动、法术释放和敌人防守。游戏中的法术包括火焰术、冰霜术和护盾术，分别对应攻击、控制和防御语义。玩家需要根据敌人出现和战斗状态及时做出手势操作，并通过 HUD 和世界空间反馈观察系统识别结果。

选择该游戏作为验证场景有三方面原因。第一，施法、移动和菜单操作能够自然映射到手势交互，适合体现动态手势的应用价值。第二，游戏系统对实时性和误触较敏感，能够检验手势命令是否足够稳定。第三，Unity 游戏原型包含输入、逻辑、UI、战斗、测试和工具链，能够作为完整工程系统支撑毕业设计论文。

3.2 功能需求分析

系统功能需求可以分为输入功能、识别功能、命令功能、游戏功能和调试功能。

输入功能方面，系统需要支持三种输入模式。Mock 模式通过键盘模拟手势和动作，便于在无摄像头或识别链路异常时进行稳定测试；Native MediaPipe 模式通过 Unity 内部摄像头链路获取手部关键点；ExternalBridge 模式通过 UDP 接收外部视觉程序输出的数据，用于接入 Python、离线视频回放或 YOLO + MediaPipe 检测链路。

识别功能方面，系统需要支持静态手势和动态手势。静态手势包括 Point、Fist、V Sign 和 Open Palm 等，用于表达指向、确认、火焰、冰霜和护盾等语义。动态手势包括左右滑动、响指、指向到握拳、身体偏移等，用于菜单切换、移动、确认和扩展动作。

命令功能方面，系统需要将识别结果转换为上层玩法可直接使用的语义命令。玩法层不应直接读取底层摄像头、关键点数组或 UDP 报文，而应读取统一的 GestureCommand。这样可以降低视觉算法变化对游戏逻辑的影响。

游戏功能方面，系统需要实现开始菜单、教学或训练入口、战斗场景、玩家移动、法术释放、敌人生成、玩家生命、护盾、防守失败、胜利或失败结算以及重新开始流程。

调试功能方面，系统需要显示当前输入模式、当前手势、动态事件、UDP 状态、桥接源、Pose 点数和性能指标等信息，便于开发、测试和答辩展示。

3.3 非功能需求分析

实时性是体感游戏的重要要求。玩家动作与游戏反馈之间的延迟应尽可能低，系统应保持稳定帧率，并避免输入处理阻塞主线程。本文通过性能监控模块统计平均 FPS、P95 帧耗时、UDP 包间隔和估算延迟等指标。

稳定性要求系统在摄像头不可用、Native MediaPipe 初始化失败或外部桥接未启动时仍能运行。为此，系统将 Mock 模式设置为安全演示路径，并通过输入路由和后端生命周期管理降低单一输入源失败对整体流程的影响。

可扩展性要求新增手势、新输入源或新玩法时不需要大规模修改已有代码。本文通过 GestureInputRouter、GestureFrame、GestureCommand 和命令历史机制隔离输入层与玩法层，使不同输入源可以向上提供统一数据结构。

可测试性要求系统核心模块能够通过自动化测试进行验证。项目将运行时代码、编辑器工具、EditMode 测试和 PlayMode 测试分离，并对场景生成、输入适配、动作识别、序列匹配和数据集校验进行测试。

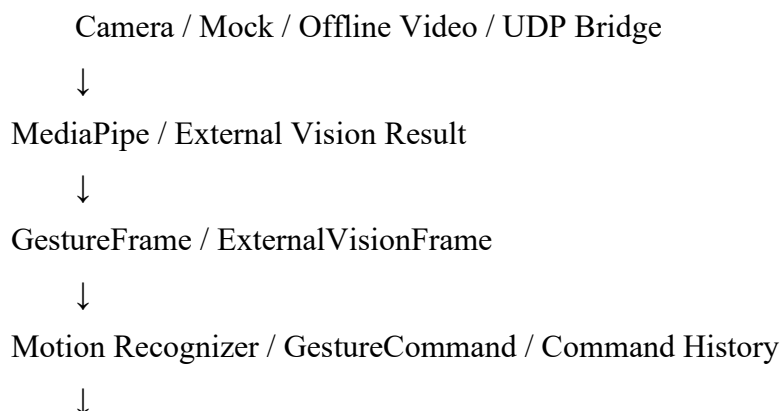
可复现性要求项目环境、依赖版本、场景配置和实验指标可以被记录。本文记录 Unity 版本、主要依赖、Build 场景、输入模式和实验指标，并通过编辑器工具生成原型场景，减少手工配置带来的不确定性。

3.4 系统总体架构

系统整体架构可以分为输入层、视觉识别层、手势抽象层、命令识别层、游戏逻辑层和反馈层。

输入层包括摄像头、Mock 输入和 UDP 外部桥接。视觉识别层包括 Native MediaPipe 手部关键点识别和外部 Python 视觉程序输出的数据。手势抽象层负责将不同来源的数据转换为统一的 GestureFrame 和 ExternalVisionFrame。命令识别层根据静态手势、动态轨迹和命令历史生成 GestureCommand。游戏逻辑层包括玩家移动、施法、敌人生成、生命和护盾等模块。反馈层包括 HUD、菜单 Overlay、动态手势反馈板和性能统计。

系统流程可概括为：



Player Motor / Spell Caster / Combat / Menu UI



HUD Feedback / World Feedback / Performance Metrics

这种分层设计的关键思想是：底层视觉输入只负责提供识别数据，上层玩法只消费语义命令，中间通过手势帧和命令层完成解耦。这样即使未来将 MediaPipe 替换为 YOLO + MediaPipe、LSTM 或 Transformer，游戏玩法模块仍可保持相对稳定。

图 3-1 给出了本文系统的总体架构。图中从输入层、视觉识别层、手势抽象层、玩法反馈层和实验数据层描述系统闭环，对应图件文件为 `unity-spell-guard/Docs/ThesisAssets/diagrams/system_architecture.svg`。

3.5 系统模块划分

Unity 工程主要模块如下：

模块	路径	主要职责
Core	Assets/Scripts/Core/	启动装配、流程控制、设置、场景上下文
Input	Assets/Scripts/Input/	输入源、手势帧、命令、动态识别、序列匹配
Player	Assets/Scripts/Player/	玩家移动、视角和法术释放
Combat	Assets/Scripts/Combat/	敌人生成、敌人行为、生命值、护盾和法术配置
UI	Assets/Scripts/UI/	菜单、HUD 和手势反馈
Diagnostics	Assets/Scripts/Diagnostics/	性能监控和实验数据导出
Editor	Assets/Editor/	原型场景生成和训练集验证工具
Tests	Assets/Tests/	EditMode 与 PlayMode 自动化测试

3.6 技术栈选型与工程支撑

本文系统并非单一脚本或概念演示，而是由 Unity 引擎、MediaPipe 手部关键点识别、UDP 外部桥接、UGUI 交互界面、Unity Test Framework 自动化测试和编辑器工具共同组成的工程型原型。技术栈选择的原则是：在保证毕业设计可演示、可复现和可扩展的前提下，优先选择实时性较好、工程集成成本可控、能够被代码和配置文件直接验证的技术。

（1）游戏引擎层。系统使用 Unity 2022.3.62f2c1 作为主要开发环境。Unity 提供组件化对象模型、场景管理、物理碰撞、UI 绘制、运行时调试和跨平台构建能力，适合快速实现体感游戏原型。项目 Build Settings 中配置了

Assets/Scenes/SpellGuardStart.unity 和 Assets/Scenes/SpellGuardPrototype.unity 两个场景，前者负责开始菜单、教学和入口流程，后者负责核心战斗演示。该配置保证独立运行时能够先进入手势友好的开始界面，再由玩家选择进入训练或战斗。

(2) 视觉输入层。系统通过 `com.github.homuler.mediapipe` 本地 `tgz` 包接入 MediaPipe Unity 插件，用于 Unity 内部摄像头输入和手部关键点识别。MediaPipe Hands 提供手部 21 个关键点，能够支撑静态手势判断和动态轨迹分析。除 Unity 内部链路外，系统还设计了 ExternalBridge 外部桥接模式，通过 UDP 接收 Python 或离线回放程序发送的视觉帧，为后续接入 YOLO + MediaPipe、离线视频测试和算法对比实验保留接口。

(3) 输入抽象层。系统没有让玩家移动、施法和菜单模块直接依赖摄像头或 MediaPipe 输出，而是通过 GestureInputRouter、GestureFrame、GestureCommand、GestureCommandHistory 等结构建立中间层。该层将不同输入源统一为游戏语义命令，使 Mock、Native MediaPipe 和 ExternalBridge 三种模式能够复用同一套玩法逻辑。这也是本文技术实现的核心：视觉识别结果只有经过命令抽象，才能稳定进入游戏系统。

(4) UI 与反馈层。系统使用 Unity UGUI 实现开始菜单、流程界面和 HUD 调试面板。SpellGuardMenuOverlay 支持手势导航和鼠标 fallback，保证演示时即使手势链路不稳定也能继续操作；DebugHud 和 MotionGestureFeedbackBoard 将输入模式、当前手势、动态事件、UDP 状态和性能信息可视化，便于调试和答辩展示。

(5) 测试与工具层。系统使用 `com.unity.test-framework` 编写 EditMode 和 PlayMode 测试，并通过 `asmdef` 将运行时代码、编辑器代码和测试代码分离为 SpellGuard.Runtime、SpellGuard.Editor、SpellGuard.EditModeTests 和 SpellGuard.PlayModeTests 等程序集。编辑器工具 CreatePrototypeScene 用于生成开始场景和战斗原型场景，TrainingDatasetValidator 用于检查训练集结构。该设计使项目具备一定工程可维护性，而不是完全依赖手工场景配置。

(6) 诊断与实验层。系统包含 GesturePerformanceMonitor 性能监控模块，用于统计 FPS、帧耗时、外部包数量、包间隔、估算延迟和命令触发次数。该模块服务于论文实验章节，可以将游戏运行状态转化为 CSV 或表格数据，支撑对实时性和交互稳定性的分析。

第 4 章 动态手势轨迹跟踪与命令识别方法

动态手势轨迹跟踪是本文系统区别于普通静态手势分类的核心部分。静态手势分类通常只需要判断某一帧图像中手部姿态属于哪一类，而体感游戏中的交互动作往往包含连续变化过程。例如，玩家向左滑动时，系统不仅要知道“画面中有一只

手”，还要判断手部在一段时间内是否发生了足够明显的横向位移；玩家做响指动作时，系统需要观察拇指和中指的距离是否经历了从接近到分离的变化；玩家从指向变为握拳时，系统需要确认这不是单帧误识别，而是一个具有交互意图的状态转换。

因此，本章将手势输入处理分为四个层次：第一层是帧级数据抽象，将 Mock、Native MediaPipe 和 ExternalBridge 的输出统一为手势帧；第二层是轨迹历史维护，用时间窗口保存近期关键点和姿态状态；第三层是动态动作识别，通过位移、速度、距离变化和状态转换判断 Swipe、Snap、PointToFist 等动作；第四层是语义命令生成，将识别结果转换为游戏逻辑可以消费的 GestureCommand。这种分层设计的目的，是让游戏系统依赖稳定的交互命令，而不是直接依赖摄像头图像、MediaPipe 关键点数组或 UDP 报文。

从工程角度看，本文采用规则驱动的动态识别方法，而不是在 Unity 内部训练和部署复杂时序深度模型。这样做主要基于三个考虑：首先，规则方法具有较强可解释性，便于在论文中说明每个动作的判定依据；其次，规则方法计算量低，适合 Unity 运行时和普通电脑环境；最后，规则方法在小规模毕业设计原型中更容易调试和复现。深度时序模型虽然在大规模数据和复杂动作识别中具有优势，但需要更多标注数据、训练成本和推理部署工作，本文将其作为后续扩展方向[13-17]。

4.1 手势数据结构设计

为了屏蔽不同输入源的差异，系统设计了统一的手势数据结构。GestureFrame 是 Unity 运行时中的帧级抽象，用于描述当前时刻手是否存在、当前静态手势、关键点信息、置信度、输入来源和时间戳等信息。无论数据来自 Mock、Native MediaPipe 还是 ExternalBridge，上层系统都可以通过统一接口读取当前手势帧。

ExternalVisionFrame 用于表示外部视觉桥接输入。该结构面向 Python 或离线视频回放链路，通常包含手部关键点、人体姿态点、手势标签、置信度、时间戳和数据源标识等字段。通过该结构，外部视觉程序可以将识别结果通过 UDP 传入 Unity，再由 Unity 侧转换为内部手势帧和命令。

GestureCommand 是面向游戏逻辑的语义命令。与 GestureFrame 相比，GestureCommand 不关心底层关键点细节，而关注“当前动作对游戏意味着什么”。例如握拳可以映射为火焰法术或确认操作，张掌可以映射为护盾或返回操作，左右滑动可以映射为菜单切换或横向移动。

此外，系统使用 GestureCommandHistory 保存近期命令，用于支持序列匹配和组合动作判断。通过命令历史，系统可以判断某组动作是否在规定时间内按正确顺序完成，从而为后续组合技或复杂交互提供基础。

这些数据结构之间的关系如表 4-1 所示。可以看到，系统并没有让玩法层直接接触原始视觉数据，而是在每一层逐步减少底层细节，增加语义稳定性。

数据结构	属层次	主要字段	作用
ExternalVisionFrame	外部视觉 桥接层	hand landmarks、pose landmarks、gesture、 confidence、 timestamp、source	接收 Python、离线视频或 YOLO + MediaPipe 链路输出
GestureFrame	Unity 手势 帧层	hand present、 static gesture、palm position、 confidence、time	统一 Mock、Native 和 ExternalBridge 三种输入源
GestureCommand	游戏命令 层	command kind、static gesture、 motion gesture、 triggered time	向玩家、战斗、菜单和 HUD 提供可消费命令
GestureCommandHistory	命令历史 层	recent commands、time window、sequence state	支持组合动作、顺序判断和超时判断

帧级抽象还有一个重要作用：处理不同坐标系之间的差异。MediaPipe 输出的关键点通常位于归一化图像坐标系，Unity 游戏逻辑更关心动作方向和相对位移，而不是具体像素位置。系统在进入动态识别前会将掌心、指尖等关键点统一到视口或归一化坐标系中，使后续阈值判断不依赖某一具体分辨率。对于 Mock 输入，系统会生成结构上等价的手势帧，虽然它没有真实关键点图像来源，但仍能提供手势类别、动态动作和时间戳，从而保证上层逻辑可以复用。

4.2 多输入源统一机制

系统支持 Mock、Native MediaPipe 和 ExternalBridge 三种输入模式，并通过 GestureInputRouter 对外提供统一数据。

Mock 模式是开发和演示中的稳定兜底路径。该模式不依赖摄像头或外部识别程序，而是通过键盘模拟手势类别、手部存在状态和动态动作。Mock 模式的值在于，当真实视觉链路受设备、权限或环境影响时，系统仍能展示完整游戏流程和交互逻辑。

Native MediaPipe 模式是 Unity 内部视觉链路。该模式通过摄像头采集图像，并使用 MediaPipe Unity 包运行手部关键点识别。识别结果被写入 Native 输入提供者，再转换为手势帧、快照和命令。该模式链路较短，适合展示普通摄像头下的实时手势识别能力。

ExternalBridge 模式用于接收外部视觉程序通过 UDP 发送的数据。该模式可以接入 Python 程序、离线视频回放或 YOLO + MediaPipe 组合链路。其优势是算法侧更灵活，便于在 Unity 外部进行实验、调参和离线对比；不足是增加了进程间通信和部署复杂度。

GestureInputRouter 的作用是根据当前模式选择对应输入提供者，并向上层暴露统一的当前快照、当前手势帧和当前手势命令。这样，玩家移动、施法、菜单和 HUD 模块不需要关心输入来自键盘模拟、Unity 摄像头还是 UDP 桥接。

多输入源统一机制的关键在于“输入源可替换，命令语义不变”。例如，向右滑动这一动作在 Mock 模式下可以由方向键模拟，在 Native MediaPipe 模式下可以由手部掌心轨迹触发，在 ExternalBridge 模式下可以由外部程序发送的关键点序列触发。虽然三条链路的数据来源不同，但最终都转换为同一种 MotionGestureType.SwipeLeftToRight，再由 GestureCommand 暴露给游戏逻辑。因此，玩家移动模块只需要处理“向右滑动命令”，不需要分别处理键盘、摄像头和 UDP 三种实现。

这种设计提高了系统的可测试性和演示稳定性。开发阶段可以使用 Mock 模式快速验证菜单、移动、施法和战斗流程；实验阶段可以使用 ExternalBridge 模式接入离线视频或训练集回放；真实演示阶段则可以切换到 Native MediaPipe 模式展示摄像头交互。如果某一种输入链路因为设备权限、光照或进程通信问题暂时不可用，系统仍然可以通过其他模式完成核心流程验证。

三种模式的职责对比如表 4-2 所示。

图 4-1 给出了三种输入源统一进入 GestureInputRouter 的链路。图中展示了 Mock、Native MediaPipe 和 ExternalBridge 如何统一转换为 GestureFrame 与 GestureCommand，对应图件文件为 unity-spell-guard/Docs/ThesisAssets/diagrams/input_pipeline.svg。

输入模式	数据来源	优势	局限	主要用途
Mock	键盘和内部模拟状态	稳定、无需摄像头、便于测试	不能证明真实视觉效果	开发、自动化测试、答辩兜底

Native MediaPipe	Unity 摄像头和 MediaPipe Hands	链 路短、演 示直观	受 摄像头、 光照、原 生包初始 化影响	实时摄像头交 互展示
ExternalBridge	UDP 外部视觉 帧、离线视频、 Python 程序	算 法侧灵 活、便于 回放和对 比	部 署链路更 复杂、存 在通信延 迟	离线实验、 YOLO + MediaPipe 扩展

4.3 静态手势识别与确认

静态手势用于表达相对稳定的姿态语义。本文系统中常用静态手势包括 Point、Fist、V Sign 和 Open Palm。Point 可用于指向和目标反馈，Fist 可用于火焰法术或确认，V Sign 可用于冰霜法术，Open Palm 可用于护盾、防御或返回菜单。

由于视觉识别结果可能出现单帧抖动，系统不能直接将每一帧的识别类别立即转换为游戏命令。为降低误触，系统引入稳定确认思想：只有当某一手势在连续若干帧或一定时间内保持稳定时，才认为该手势有效。这样可以减少短暂误识别对游戏逻辑的影响。

在游戏映射中，静态手势和上下文共同决定最终命令。例如，在战斗状态下握拳主要表示释放火焰术；在菜单状态下握拳、响指或 PointToFist 可以表示确认当前选项；张掌在战斗状态下可以表示护盾，在菜单中可以表示返回或取消。通过上下文映射，系统能够复用有限的手势表达多种游戏语义。

静态手势的识别结果在系统中并不直接等价于最终游戏行为，而是先形成姿态语义，再由当前流程状态解释。例如，同样的 Fist 在训练流程中表示“完成握拳训练步骤”，在战斗流程中表示“释放火焰术”，在菜单流程中则可以表示“确认当前选项”。这种上下文解释机制可以减少手势数量，降低玩家学习成本，也符合体感游戏中“少量动作、多种语义”的设计原则。

为了降低静态手势误触，系统使用三类约束。第一是手部存在约束，当当前帧未检测到手或置信度低于阈值时，不生成有效命令。第二是稳定时间约束，只有姿态在连续帧中保持相对稳定，才认为该手势可以被消费。第三是上下文约束，即某些手势只在特定游戏状态下生效，例如防御手势只在战斗中触发护盾，菜单状态下则优先解释为返回或取消。通过这些约束，系统避免将单帧噪声、手部进出画面或玩家自然调整姿势误认为明确操作。

表 4-3 给出了本文原型中静态手势到游戏语义的主要映射关系。

静态 手势	基础 语义	菜单状态	训练状态	战斗状态
Point	指向、选择	选中或辅助确认	检测手部存在与指向训练	可作为瞄准或移动辅助
Fist	握拳、释放	确认当前选项	完成握拳训练	释放火焰法术
V Sign	双指、冰霜	可作为扩展选项	完成 V Sign 训练	释放冰霜法术
Open Palm	张掌、防御	返回或取消	完成张掌训练	释放护盾或防御

这种映射不是写死在视觉识别层，而是在玩法层根据流程状态解释命令。视觉层只负责回答“玩家做了什么动作”，玩法层负责回答“这个动作在当前状态下意味着什么”。这一点对系统可扩展性很重要：当后续新增法术、关卡或菜单功能时，可以优先调整命令映射，而不必重写底层手势识别算法。

4.4 动态手势轨迹识别

动态手势识别基于连续帧中的关键点变化。本文系统主要采用规则方法，即在固定时间窗口内观察关键点位置、距离和手势状态变化是否满足预设条件。

本文项目中动态识别分别由 `NativeMotionGestureRecognizer` 和 `ExternalMotionGestureRecognizer` 实现。两者共享相同的核心思想：先将连续输入帧加入历史队列，再从历史窗口中取最早样本和最新样本计算位移、速度、漂移和状态变化，最后根据阈值和冷却时间判断是否生成动态动作事件。`Native` 链路的数据来自 `NativeMediapipeGestureProvider`，`External` 链路的数据来自 `ExternalGestureBridgeProvider` 接收到的 `ExternalVisionFrame`。这种设计使两条输入链路能够复用同一套动态动作语义。

设某一时刻的手部样本为：

$$s_i = (t_i, p_i, g_i, q_i)$$

其中， t_i 表示时间戳， $p_i = (x_i, y_i)$ 表示掌心或回退指针在视口坐标系中的位置， g_i 表示当前静态手势类型， q_i 表示置信度或可见度。系统维护一个长度约为 T_h 的历史窗口：

$$H_t = \{s_i \mid 0 \leq t - t_i \leq T_h\}$$

当前项目中，`Native` 和 `External` 动态识别器的历史窗口 `historySeconds` 均设置为 0.7 秒。若历史样本数量少于 3，则不触发动态识别，以避免单帧或双帧抖动造成误判。

Swipe 动作主要根据掌心或关键点中心的横向位移、纵向漂移、动作持续时间和速度阈值判断。如果在规定时间内横向位移超过阈值，且垂直漂移未过大，则可识别为向左或向右滑动。该动作可用于菜单切换、横向移动或方向选择。

以滑动动作为例，取历史窗口中的首帧样本 s_0 和末帧样本 s_n ，定义：

$$\begin{aligned}\Delta x &= x_n - x_0 \\ \Delta y &= y_n - y_0 \\ \Delta t &= \max(t_n - t_0, \varepsilon) \\ v_x &= |\Delta x| / \Delta t \\ v_y &= |\Delta y| / \Delta t\end{aligned}$$

其中 ε 是防止除零的极小值。横向滑动成立条件为：

$$\begin{aligned}|\Delta x| &\geq D_{\text{swipe}} \\ |\Delta y| &\leq B_{\text{swipe}} \\ v_x &\geq V_{\text{swipe}} \\ t_n - t_{\text{lastSwipe}} &\geq C_{\text{swipe}}\end{aligned}$$

纵向滑动成立条件为：

$$\begin{aligned}|\Delta y| &\geq D_{\text{swipe}} \\ |\Delta x| &\leq B_{\text{swipe}} \\ v_y &\geq V_{\text{swipe}} \\ t_n - t_{\text{lastSwipe}} &\geq C_{\text{swipe}}\end{aligned}$$

当前项目中， D_{swipe} 对应 `swipeMinDistance`，默认值为 0.09； B_{swipe} 对应 `swipeMaxVerticalDrift`，默认值为 0.22； V_{swipe} 在代码中为 0.2； C_{swipe} 对应 `swipeCooldownSeconds`，默认值为 0.28 秒。若横向滑动成立且 $|\Delta x| \geq |\Delta y|$ ，则根据 Δx 符号输出 `SwipeLeftToRight` 或 `SwipeRightToLeft`；否则根据 Δy 符号输出 `SwipeBottomToTop` 或 `SwipeTopToBottom`。

Snap 动作主要根据拇指和中指之间的距离变化判断。当两指距离先接近，再在短时间内明显分离，且满足冷却约束时，可以识别为响指动作。该动作适合作为确认或特殊触发动作。

响指动作使用拇指指尖和中指指尖距离判断。设第 i 帧拇指指尖为 u_i ，中指指尖为 m_i ，二者距离为：

$$d_i = \|u_i - m_i\|$$

系统先判断是否进入预备状态：

$$d_i \leq D_{\text{close}}$$

当进入预备状态后，如果在最大持续时间 T_{snap} 内检测到：


```

d_j >= D_release
t_j - t_prime <= T_snap
t_j - t_lastSnap >= C_snap

```

则触发 Snap。当前项目中，D_close 对应 snapCloseDistance，默认值为 0.09；D_release 对应 snapReleaseDistance，默认值为 0.14；T_snap 对应 snapMaxDuration，默认值为 0.35 秒；C_snap 对应 snapCooldownSeconds，默认值为 0.45 秒。该设计要求响指动作具有“先靠近、后释放”的过程，而不是只看某一帧的距离。

PointToFist 动作关注手势类别的状态变化。当系统检测到玩家先处于指向状态，随后在规定时间内转换为握拳状态，可以认为玩家完成了从瞄准到施法的动作。这类状态转换比单纯握拳更能表达“准备并释放”的交互语义。

PointToFist 动作由 Native 链路中的静态手势状态变化触发。设上一次稳定观察到的手势为 g_prev，当前手势为 g_now，上一次变化时间为 t_prev，当前时间为 t_now，掌心位移为 ||p_now - p_prev||。当满足：

```

g_prev = Point
g_now = Fist
T_hold <= t_now - t_prev <= T_transition
||p_now - p_prev|| <= D_transition
t_now - t_lastTransition >= C_transition

```

则输出 PointToFist。当前项目中，T_hold 对应 pointHoldMinDuration，默认值为 0.08 秒；T_transition 对应 gestureTransitionMaxDuration，默认值为 0.4 秒；D_transition 对应 gestureTransitionMaxTravel，默认值为 0.18；C_transition 对应 gestureTransitionCooldownSeconds，默认值为 0.45 秒。该规则能够减少单纯握拳带来的误触，因为它要求玩家先做出指向，再在短时间内切换为握拳。

BodyShift 动作可通过人体姿态关键点或中心点偏移进行判断，用于表达身体向左或向右移动的意图。当前系统将其作为外部桥接链路和后续扩展方向之一。人体活动识别和骨架序列建模研究表明，关键点序列能够在不直接依赖原始图像纹理的情况下表达动作结构，因此也适合用于体感交互中的身体移动判断[23-24]。

BodyShift 动作由 External 链路根据人体姿态点判断。系统从外部帧的 poseLandmarks 中读取左右肩和左右髋关键点，并计算身体中心：

```
c_i = (shoulderCenter_i + hipCenter_i) / 2
```

若姿态点可见度低于阈值，则该帧不参与身体偏移判断。对于历史窗口首尾样本，若满足：

```

|Δx_body| >= D_body
|Δy_body| <= B_body

```

$|\Delta x_{\text{body}}| / \Delta t \geq V_{\text{body}}$

$t_n - t_{\text{lastBodyShift}} \geq C_{\text{body}}$

则根据 Δx_{body} 符号输出 BodyShiftLeft 或 BodyShiftRight。当前项目中， D_{body} 对应 bodyShiftMinDistance，默认值为 0.1； B_{body} 对应 bodyShiftMaxVerticalDrift，默认值为 0.12；速度阈值为 0.28；冷却时间 C_{body} 默认值为 0.45 秒；姿态可见度阈值 minPoseVisibility 默认值为 0.45。

动态动作识别流程可概括为：连续输入关键点序列，加入历史队列；在时间窗口内计算位移、速度、距离或状态变化；判断是否满足动作阈值；若满足条件且未处于冷却期，则生成对应 MotionGestureEvent 或 GestureCommand；最后由玩法系统消费该命令。

图 4-2 给出了动态手势识别流程。图中从输入帧、历史窗口、特征计算、规则判断、冷却过滤到命令生成描述动态动作识别过程，对应图件文件为 unity-spell-guard/Docs/ThesisAssets/diagrams/motion_recognition_flow.svg。

与直接将每一帧识别结果作为游戏输入相比，该流程增加了历史窗口、阈值判断和冷却过滤三个环节。历史窗口用于观察动作过程，阈值判断用于保证动作幅度和方向有效，冷却过滤用于避免同一动作在短时间内重复触发。最终生成的 GestureCommand 只包含游戏层需要的动作类型、置信度、触发时间等信息，从而降低底层视觉数据对玩法模块的影响。

表 4-4 汇总了当前动态手势识别中使用的主要参数。参数值并不直接绑定某一种摄像头或玩家动作，而是通过 GestureRecognitionProfile 等配置入口集中管理，便于根据摄像头距离、画面裁剪范围和演示环境进行调整。

参数	含义	当	
		前默认值	作用
historySeconds	历史窗口长度	0.70 s	限制参与动态识别的连续帧范围
swipeMinDistance	滑动最小位移	0.09	过滤幅度过小的横向或纵向移动
swipeMaxVerticalDrift	滑动允许漂移	0.22	避免斜向抖动误判为水平滑动
swipeCooldownSeconds	滑动冷却时间	0.28 s	防止同一次滑动重复触发
snapCloseDistance	响指预备距离	0.09	判断拇指与中指是否进入接近状态
snapReleaseDistance	响指释放距离	0.14	判断两指是否从接近状态

	放距离		态快速分离
snapMaxDuration	响指最	0.35 s	保证响指动作具有短时
	大持续时间		动态特征
gestureTransitionMaxDuration	状态转	0.40 s	限制 Point 到 Fist 的转换
	换最大时长		窗口
gestureTransitionMaxTravel	状态转	0.18	避免大幅移动时误触发
	换最大位移		PointToFist
bodyShiftMinDistance	身体偏	0.10	判断身体中心是否发生
	移最小位移		有效横向偏移
minPoseVisibility	姿态点	0.45	过滤姿态关键点置信度
	最低可见度		不足的帧

上述规则方法的优势在于实时性强、可解释性高、调参成本低，适合毕业设计原型和答辩演示场景。其局限在于参数对摄像头距离、玩家动作幅度和画面坐标归一化方式较敏感。如果玩家动作幅度明显小于阈值，可能出现漏触发；如果背景、光照或关键点估计导致坐标抖动，可能出现误触发。因此系统同时采用三类抑制策略：第一，要求历史窗口内样本数量满足最低条件，避免单帧噪声直接触发；第二，使用位移、漂移、速度和状态转换的组合条件，而不是只看单个特征；第三，在每类动态动作后设置冷却时间，避免一个动作被连续消费多次。

4.5 命令历史与序列匹配

为了支持组合动作和复杂交互，系统设计了命令历史与序列匹配机制。`GestureCommandHistory` 保存一段时间内发生的手势命令，`GestureSequenceMatcher` 根据顺序、时间窗口和命令类型判断某个动作序列是否成立。

例如，未来可以定义 `Point → Fist → Snap` 作为某种高级法术释放序列。系统需要判断玩家是否先做出指向动作，再在限定时间内握拳，最后完成响指。如果顺序错误、超时或中间出现不允许的命令，则该序列不成立。

设命令历史为：

$$Q = \{c_1, c_2, \dots, c_n\}$$

其中每个命令 c_i 包含命令类型、静态手势或动态动作、触发时间 t_i 。对于目标序列：

$$P = \{p_1, p_2, \dots, p_m\}$$

系统从历史队列中按时间顺序寻找与 P 对应的命令。如果存在子序列 $\{c_a, \dots, c_b\}$ 满足命令类型依次匹配，且：

$$t_b - t_a \leq T_{\text{sequence}}$$

$$t_i < t_j, i < j$$

则认为序列匹配成功；如果出现顺序不一致、间隔超时或命令类型不符合模式要求，则匹配失败。该设计使系统能够在不改变底层视觉识别器的情况下扩展组合动作，例如将“指向、握拳、响指”解释为更高阶的施法语义。

序列匹配机制的意义在于，它使系统从“单个手势触发单个命令”扩展到“多个动作组合表达复杂语义”。这对于体感游戏中的组合技、菜单快捷操作和高级交互具有扩展价值。同时，序列匹配仍然工作在 `GestureCommand` 层，而不是直接读取关键点数组，因此它与 `Mock`、`Native MediaPipe` 和 `ExternalBridge` 三种输入模式保持解耦。

4.6 本章小结

本章围绕动态手势轨迹跟踪与命令识别方法，说明了系统如何将视觉识别结果逐步转换为 Unity 游戏可以消费的交互命令。首先，系统通过 `ExternalVisionFrame`、`GestureFrame`、`GestureCommand` 和 `GestureCommandHistory` 构建分层数据结构，使原始视觉数据、统一手势帧、游戏语义命令和命令历史相互分离。其次，系统通过 `GestureInputRouter` 统一 `Mock`、`Native MediaPipe` 和 `ExternalBridge` 三种输入模式，使不同输入来源能够复用同一套玩家移动、施法、菜单和 HUD 逻辑。

在识别方法上，本文采用基于时间窗口、位移阈值、速度约束、指尖距离变化、状态转换和冷却时间的规则方法，分别识别 `Swipe`、`Snap`、`PointToFist` 和 `BodyShift` 等动态动作。该方法虽然不属于大规模深度学习时序模型，但具备实时性强、可解释、易调参和易复现的特点，适合本文面向体感游戏原型的工程目标。最后，系统通过命令历史与序列匹配机制，为后续组合动作和高级交互保留扩展空间。上述方法共同构成了从“视觉帧”到“游戏命令”的核心转换链路。

第 5 章 Unity 游戏原型系统实现

5.1 工程环境与目录结构

本文系统基于 Unity 2022.3.62f2c1 开发。该版本属于 Unity 2022 LTS 系列，适合毕业设计这类需要稳定编辑器环境和长期维护的项目[25]。项目使用 Unity Hub 打开 `unity-spell-guard/` 子工程，运行入口为 `Assets/Scenes/SpellGuardStart.unity`。`Build Settings` 中已配置开始场景和战斗原型场景，顺序为 `SpellGuardStart.unity` 在前、`SpellGuardPrototype.unity` 在后。因此独立运行时首先进入开始菜单，再根据玩家选择进入训练或战斗流程。

项目依赖由 `Packages/manifest.json` 管理，主要包括以下几类。第一类是视觉识别依赖，`com.github.homuler.mediapipe` 以本地 `tgz` 形式接入 `MediaPipe Unity` 插件，用于摄像头图像处理和手部关键点识别[4]。第二类是测试依赖，`com.unity.test-framework` 用于编写和运行 `EditMode`、`PlayMode` 自动化测试[26]。第三类是界面与

Unity 常规模块，com.unity.ugui 用于菜单、HUD 和运行时反馈，com.unity.timeline、com.unity.visualscripting、com.unity.modules.physics、com.unity.modules.video 等用于补充 Unity 常规功能。第四类是开发辅助依赖，例如 IDE 支持包、本地工具包和 Unity MCP 插件等，这些依赖主要影响本地开发和复现环境。

从工程组织上看，项目使用 asmdef 明确划分程序集边界，包括 SpellGuard.Runtime.asmdef、SpellGuard.Editor.asmdef、SpellGuard.EditModeTests.asmdef 和 SpellGuard.PlayModeTests.asmdef。这种划分使运行时代码、编辑器工具和测试代码相互隔离，避免测试或编辑器专用逻辑进入运行时程序集，也便于后续定位编译错误和维护模块依赖。

工程目录按照职责划分为 Core、Input、Player、Combat、UI、Diagnostics、Editor 和 Tests 等模块。Core 模块负责启动装配、流程控制、设置和场景上下文；Input 模块负责手势输入、路由、帧数据、命令抽象、动态识别和序列匹配；Player 模块负责玩家移动和施法；Combat 模块负责敌人生成、敌人行为、生命、护盾和法术配置；UI 模块负责菜单、HUD 和手势反馈；Diagnostics 模块负责性能监控和实验数据导出；Editor 模块负责原型场景生成和训练集校验；Tests 模块负责 EditMode 与 PlayMode 自动化测试。

表 5-1 给出了主要技术栈与其在系统中的作用。

图 5-1 给出了 Unity 原型系统的模块结构。图中展示了 Core、Input、Player、Combat、UI、Diagnostics、Editor/Tests 等模块及其依赖关系，对应图件文件为 unity-spell-guard/Docs/ThesisAssets/diagrams/unity_module_structure.svg。

技术或组件	项目中的作用	对论文的支撑
Unity 2022.3.62f2c1	游戏引擎、场景、组件、运行时逻辑	支撑体感游戏原型实现
MediaPipe Unity Plugin	摄像头手部关键点识别	支撑视觉输入和手势关键点获取
UDP ExternalBridge	外部视觉帧接入、离线回放、算法扩展	支撑 YOLO + MediaPipe 融合路线与外部实验
Mock 输入源	无摄像头开发、测试、答辩兜底	支撑系统稳定性和可演示性
UGUI	菜单、HUD、运行时反馈	支撑游戏流程和调试可视化
Unity Test Framework	EditMode / PlayMode 自动化测试	支撑可测试性和工程可靠性
asmdef 程序集	Runtime / Editor / Tests	支撑工程模块化

Editor 工具	场景生成、训练集校验	支撑可复现性和数据准备
GesturePerformanceMonitor	FPS、延迟、命令统计	支撑实验章节的数据采集

5.2 场景设计与启动装配

项目包含两个核心场景：`SpellGuardStart.unity` 和 `SpellGuardPrototype.unity`。前者是开始菜单场景，包含主菜单、教学、设置、训练入口和战斗入口等内容；后者是主要战斗原型场景，包含第一人称视角、玩家对象、敌人生成、法术释放、HUD 和战斗流程。

项目提供 `CreatePrototypeScene` 编辑器工具，用于生成原型场景和必要对象。该工具能够创建地面、通道、祭坛、墙体、灯光、玩家对象、输入对象、玩法对象、UI 和反馈对象。与完全手工搭建场景相比，编辑器生成工具提高了场景复现能力，也便于通过测试验证关键对象是否存在。

`SpellGuardSceneContext` 是场景级依赖容器，负责集中保存输入、玩家、战斗和 UI 等关键引用，并提供自动绑定和完整性检查能力。`SpellGuardBootstrap` 负责在运行时进行启动装配，包括自动绑定缺失引用、校验场景有效性、配置玩家移动、施法、敌人生成、流程控制、输入后端、HUD 和反馈板等。

启动流程大致如下：首先场景加载后由 `Bootstrap` 触发依赖绑定和检查；随后根据当前输入模式配置对应输入后端；接着初始化玩家、战斗和 UI 模块；最后进入菜单、训练或战斗流程。该过程保证了系统各模块之间的引用关系在运行开始前得到确认。

场景启动装配的设计重点是降低手工配置错误。`Unity` 场景中的对象引用较多，如果完全依赖 `Inspector` 手动拖拽，容易在迭代过程中出现引用丢失、组件未挂载或场景对象命名变化导致的运行错误。`SpellGuardBootstrap` 通过集中检查关键组件是否存在，在运行开始阶段尽早暴露问题。例如输入路由、玩家控制器、流程控制器、HUD、反馈板和性能监控器都属于启动时需要确认的对象。若某些引用缺失，系统会尝试通过场景上下文或对象搜索进行自动绑定；如果仍无法满足运行条件，则通过日志提示定位问题。

原型场景还采用了“开始场景 + 战斗场景”的双场景结构。开始场景负责承载主菜单、设置、训练入口和模式选择，降低玩家或评审进入系统时的理解成本；战斗场景负责承载第一人称移动、敌人生成、法术释放和 HUD 调试反馈。两者通过流程控制器和 `Build Settings` 中的场景顺序衔接，使系统既可以作为完整游戏流程运行，也可以在开发阶段直接打开战斗场景调试核心玩法。

表 5-2 给出了启动装配过程中的主要对象和职责。

对象或组件	所属模块	职责
-------	------	----

SpellGuardBootstrap	Core	运行时依赖绑定、场景检查、输入与 HUD 装配
SpellGuardSceneContext	Core	保存场景级关键引用，降低跨对象查找成本
GestureInputRouter	Input	根据当前模式选择输入源并输出统一命令
SpellGuardFlowController	Core/UI	管理菜单、训练、战斗、结果等流程状态
FpsGestureMotor	Player	根据手势命令驱动玩家移动
GestureSpellCaster	Player/Comb	将手势命令解释为法术释放
at		
DebugHud	UI	显示输入模式、手势状态、桥接状态和性能信息
GesturePerformanceMonitor	Diagnostics	采集 FPS、帧耗时、命令数和延迟指标

5.3 输入系统实现

输入系统位于 Assets/Scripts/Input/，是本文项目的核心模块之一。

GestureInputProviderBase 为不同输入源提供统一基类，暴露当前快照、当前动态动作、当前手势帧、当前手势命令和近期命令历史等数据。

MockGestureInputProvider 通过键盘模拟手势和手部移动，可在没有摄像头或识别失败时稳定运行。该模式支持切换手部存在状态、切换 Point、Fist、V Sign、Open Palm 等手势，并可通过方向键模拟滑动动作。

NativeMediapipeGestureRunner 和 NativeMediapipeGestureProvider 共同组成 Unity 内部 MediaPipe 链路。前者负责摄像头输入和 MediaPipe 图运行，后者保存当前识别状态和生成的手势数据。为避免原生链路初始化失败影响整体流程，系统通过输入模式切换和后端生命周期管理控制其启动时机。

ExternalGestureBridgeProvider 和 UdpGestureReceiver 组成外部桥接链路。外部程序通过 UDP 发送视觉帧，Unity 接收后转换为内部手势帧和命令。该路径适合接入 Python 视觉处理、离线视频回放和 YOLO + MediaPipe 对比实验。

输入系统在实现上采用“提供者 + 路由器”的结构。各输入源只负责维护自己的识别状态和帧数据，GestureInputRouter 则负责选择当前启用的输入源，并向上层暴露统一接口。这样可以避免玩家、战斗和 UI 模块直接依赖某个具体输入实现。例如，玩家移动模块只读取当前 GestureCommand，不需要知道该命令来自键盘模拟、MediaPipe 摄像头还是 UDP 外部桥接。

在 Native MediaPipe 模式下，系统需要处理摄像头权限、MediaPipe 图初始化和 Unity 主线程更新之间的关系。运行时并不在所有情况下立即启动摄像头链路，而是根据当前输入模式和场景状态控制其生命周期。这样可以减少不必要的资源占用，也避免在没有摄像头或权限异常时影响 Mock 模式和菜单流程。Native 链路输出手部关键点后，系统会计算掌心位置、当前静态手势和动态动作候选，再转换为统一手势帧。

在 ExternalBridge 模式下，Unity 侧通过 UDP 接收外部视觉帧。外部帧中通常包含时间戳、数据源标识、手部关键点、人体姿态点、手势标签和置信度。Unity 接收后并不直接执行玩法逻辑，而是先由 ExternalGestureBridgeProvider 将外部结构转换为内部手势帧，再由动态识别器和命令层生成可消费命令。这种设计为离线视频回放、Python 侧算法调试和 YOLO + MediaPipe 前置检测提供了接口。

表 5-3 汇总了输入系统主要类的职责。

类或组件	职责	输出
GestureInputProviderBase	输入源基类，定义 统一状态接口	快照、手势帧、命令 历史
MockGestureInputProvider	通过键盘模拟手势 和动态动作	Mock 手势命令
NativeMediapipeGestureRunner	运行摄像头和 MediaPipe 图	原生关键点识别结果
NativeMediapipeGestureProvider	保存 Native 识别状 态并生成帧	Native 手势帧
UdpGestureReceiver	接收外部 UDP 视 觉帧	ExternalVisionFrame
ExternalGestureBridgeProvider	转换外部帧并识别 动态动作	ExternalBridge 手势命 令
GestureInputRouter	选择当前输入模式 并统一暴露状态	当前快照、当前命令

5.4 玩家移动与施法实现

玩家移动由 FpsGestureMotor 实现。该模块读取当前手势命令，根据左右滑动、身体偏移或其他移动语义控制玩家在第一人称场景中的运动。相比直接使用键盘输入，手势移动更强调动作与空间反馈之间的对应关系。

法术释放由 GestureSpellCaster 实现。系统将 Fist、V Sign 和 Open Palm 等手势映射为火焰、冰霜和护盾语义。火焰法术主要用于攻击敌人，冰霜法术用于控制或

冻结敌人，护盾法术用于防御或减少伤害。施法模块读取输入路由提供的命令，判断当前是否满足施法条件，并将结果作用到敌人或玩家状态上。

这种设计将“手势是什么”和“游戏中做什么”分离。输入层负责识别动作，施法层负责根据游戏状态解释动作含义。例如，同样的握拳动作在菜单中可以表示确认，在战斗中可以表示释放火焰术。

玩家移动并不采用连续模拟摇杆的方式，而是将部分手势解释为离散移动意图。这样做可以降低体感输入抖动对第一人称控制的影响。左右滑动或身体偏移可以触发横向移动，Point 或 OpenPalm 等稳定手势可以作为前进、后退或训练动作的触发条件。为了避免静态手势长时间保持时不断重复移动，移动模块会记录上一次已消费的手势状态，只有当手势发生变化、冷却结束或动态动作重新触发时，才执行新的移动响应。

施法系统采用“手势语义 + 法术效果”的组合。GestureSpellCaster 读取输入路由中的命令后，根据当前战斗状态判断是否释放法术。火焰法术通常对应攻击型反馈，用于对敌人造成伤害；冰霜法术对应控制型反馈，用于冻结或减缓敌人；护盾法术对应防御型反馈，用于提升玩家存活能力。每类法术既有输入触发条件，也有作用对象和反馈效果。这样的设计让手势识别结果能够通过战斗系统体现出来，而不是停留在 HUD 上的文字变化。

表 5-4 给出了主要手势命令与玩法行为之间的对应关系。

命令			
手势或动态动作	类型	玩家移动	法术或流程语义
SwipeLeftToRight / SwipeRightToLeft	Motion	横向移动或 菜单切换	方向选择
SwipeBottomToTop / SwipeTopToBottom	Motion	前进或后退	训练动作或位移 反馈
Point	StaticPose	可作为前进 或瞄准辅助	指向、选择、训练步骤
Fist	StaticPose	无直接位移	火焰法术、确认
V Sign	StaticPose	无直接位移	冰霜法术
Open Palm	StaticPose	可作为后退 或停止语义	护盾、防御、返回
Snap / PointToFist	Motion	无直接位移	确认、特殊触发 或扩展施法

5.5 战斗与流程控制实现

战斗系统主要由 EnemySpawner、SimpleEnemyController、PlayerHealth 和 GameFlowManager 组成。EnemySpawner 负责按照规则生成敌人，形成持续压力；SimpleEnemyController 负责敌人移动、受击和状态变化；PlayerHealth 负责玩家生命值和护盾；GameFlowManager 负责胜利、失败和重开等流程。

流程控制由 SpellGuardFlowController 负责。系统包含开始菜单、教学、设置、训练、战斗、暂停和结果等状态。玩家可以通过手势或鼠标 fallback 操作进入不同流程。对于答辩演示而言，完整流程应覆盖主菜单、教学或训练、战斗、结算和返回菜单，体现系统不仅是识别 demo，而是可运行的游戏原型。

敌人生成模块负责为战斗场景提供持续交互对象。如果没有敌人和胜负反馈，手势施法只能停留在输入演示层面，难以证明手势命令已经进入游戏逻辑。EnemySpawner 按配置在场景中生成敌人，SimpleEnemyController 控制敌人靠近玩家、响应受击和状态变化。玩家释放火焰或冰霜法术，战斗模块根据命中结果更新敌人状态；敌人接近或攻击玩家时，PlayerHealth 更新玩家生命和护盾状态。

流程控制器的作用是把分散模块组织成完整体验。开始菜单提供进入训练、战斗和设置的入口；训练流程用于引导玩家依次尝试核心手势；战斗流程用于验证手势移动、施法和防御是否能够与敌人交互；结果流程用于展示胜负、得分或施法统计，并提供重新开始或返回菜单的入口。通过这些状态，系统形成了“进入系统、学习手势、进行战斗、查看结果”的闭环。

表 5-5 给出了主要流程状态及其功能。

状态	主要内容	手势交互重点
Start	主菜单、训练入口、战斗入口、设置入口	菜单导航、确认、返回
Menu		
Setting	输入模式选择、调试配置	Mock / Native / ExternalBridge 切换
s		
Traini	核心手势练习和反馈	Point、Fist、V Sign、OpenPalm、
ng		Swipe、Snap
Comba	第一人称战斗、敌人生成、施法与防御	移动、火焰、冰霜、护盾
t		
Result	胜负结果、流程统计、返回入口	重开或返回菜单
s		

5.6 UI 与调试反馈实现

UI 与反馈模块主要包括 DebugHud、SpellGuardMenuOverlay 和 MotionGestureFeedbackBoard。SpellGuardMenuOverlay 用于开始菜单和流程界面，支持手势导航和鼠标 fallback。菜单布局根据屏幕安全区域和宽高比例进行适配，保证在不同窗口尺寸下仍具有可操作性。

DebugHud 用于显示当前输入模式、当前手势、动态事件、桥接源、UDP 状态、Pose 点数和性能信息等。它在开发和答辩中具有重要作用，因为它能够将底层识别状态可视化，使观察者理解系统当前是否识别到手、处于何种输入模式、是否收到外部桥接数据以及是否触发动态动作。

MotionGestureFeedbackBoard 用于世界空间中的手势反馈，能够在游戏场景中显示动态动作事件和识别状态。该设计增强了调试体验，也让手势识别结果更自然地融入游戏场景。

UI 设计同时服务于玩家操作和论文验证。对于玩家而言，菜单和训练界面需要明确当前可执行动作和流程状态；对于论文和答辩而言，HUD 需要把底层输入状态、桥接状态和性能指标可视化。DebugHud 因此不仅是调试工具，也是证明系统链路有效的重要证据。当 HUD 显示当前输入模式、当前手势、最近动态事件、UDP 包数量和性能数据时，观察者可以确认视觉链路已经进入 Unity，并被上层系统消费。

ExternalBridge 状态显示尤其重要。外部桥接链路比 Native 链路多了 Python 程序、UDP 通信和时间戳对齐等环节，如果 Unity 侧没有可视化状态，就很难判断数据是否真正到达游戏系统。因此 HUD 中显示 source、packet count、last packet age、estimated latency 和 last motion event 等字段，用于区分“外部程序未启动”“Unity 未收到包”“收到包但未触发动作”等不同情况。

此外，菜单保留鼠标 fallback 并不削弱体感交互主题，反而提高了演示可靠性。体感系统容易受到光照、摄像头权限和现场环境影响，如果所有流程都完全依赖手势，一旦识别链路临时异常，答辩演示就可能被阻塞。鼠标 fallback 让演示者能够继续进入训练、战斗和结果界面，而 HUD 仍然可以展示当前手势链路状态。

5.7 运行界面与截图证据

为了使系统实现结果能够被直观验证，本文对当前 Unity 原型的主要运行界面进行了截图归档。截图文件统一保存在 unity-spell-guard/Docs/ThesisAssets/screenshots/ 目录下，覆盖开始界面、玩法说明、实际战斗、摄像头校准、开发者实验室和自定义手势验证等环节。这些截图与前文的系统模块相互对应，能够从界面层面说明系统已经具备可运行的菜单流程、手势说明、输入校准、游戏交互和调试验证能力。

图 5-2 展示系统开始界面，对应文件为 `screenshots/start_menu_latest.png`。该界面用于进入玩法说明、训练、战斗或设置流程，是完整游戏闭环的入口。与单独的识别脚本不同，开始界面说明系统已经具备面向玩家的流程组织，而不是只停留在控制台或调试窗口输出。

图 5-3 展示玩法说明界面，对应文件为 `screenshots/gameplay_instruction.png`。该界面用于向玩家说明核心手势与游戏行为之间的关系，例如滑动、握拳、张掌和特殊动作对应的移动、施法或防御语义。玩法说明界面在系统中承担教学作用，也从交互设计角度降低玩家学习成本。

图 5-4 展示实际游玩界面，对应文件为 `screenshots/combat_gameplay.png`。该截图体现了第一人称战斗场景、HUD 信息和游戏反馈，说明手势输入最终并非只生成标签，而是能够进入玩家移动、法术释放、敌人交互和战斗反馈流程。该图可作为第 5 章系统实现和第 6 章功能验证之间的界面证据。

图 5-5 展示摄像头校准界面，对应文件为 `screenshots/camera_calibration.png`。摄像头校准用于在进入 Native MediaPipe 链路前检查摄像头画面、输入状态和玩家站位，使系统能够在不同设备和现场环境下进行基本调试。该界面也体现了体感系统对运行环境的依赖，以及在工程实现中需要提供可视化校准入口。

图 5-6 展示开发者实验室界面，对应文件为 `screenshots/developer_lab.png`。开发者实验室面向调试和实验，集中放置输入模式切换、调试状态或实验辅助功能。其意义在于把 Mock、Native MediaPipe 和 ExternalBridge 等输入链路的状态显式暴露出来，便于开发阶段快速定位问题，也便于答辩演示时说明当前系统运行在哪一种输入模式下。

图 5-7 展示自定义手势验证界面，对应文件为 `screenshots/custom_gesture_validation.png`。该界面用于验证自定义手势模板、当前手势状态和识别反馈，体现了系统在固定规则手势之外的扩展能力。通过该界面，用户可以观察手势模板与实时输入之间的匹配情况，为后续扩展更多动作语义提供工程基础。

上述截图形成了从“进入系统”到“理解玩法”、再到“实际战斗”和“调试验证”的界面证据链。结合第 4 章的手势识别方法和第 6 章的实验数据，可以更完整地说明本文系统已经形成从视觉输入、命令映射、游戏行为到反馈展示的运行闭环。

5.8 性能监控与实验导出

为了支撑论文实验章节，项目设计了 `GesturePerformanceMonitor` 性能监控模块。该模块可统计平均 FPS、最小 FPS、平均帧耗时、P95 帧耗时、外部 UDP 包数量、平均包间隔、估算延迟、静态命令数量、动态命令数量和不同动态动作次数。

性能监控的意义在于将“系统感觉流畅”转化为可以记录和分析的数据。对于 ExternalBridge 模式，系统还可以利用外部帧 timestamp 和 Unity 接收时间估算链路延迟。后续实验可分别在 Mock、Native MediaPipe 和 ExternalBridge 模式下运行固定时长，导出 CSV 并比较不同模式的帧率、延迟和命令触发情况。

性能数据导出采用 CSV 格式，便于在论文中整理为表格，也便于后续使用 Excel 或脚本进行统计。导出字段包括 session_id、mode、source、elapsed_seconds、average_fps、p95_frame_ms、external_packets、avg_packet_interval_ms、avg_estimated_latency_ms、static_commands、motion_commands 以及各类动态动作触发次数。对于 Mock 和 Native 模式，外部包和链路延迟字段通常不适用；对于 ExternalBridge 模式，这些字段可以反映 UDP 回放或外部视觉程序的输入吞吐。

除性能监控外，系统还实现了 DemoRunRecorder 用于记录演示流程。它关注的不是每帧性能，而是一次演示是否经过开始菜单、训练、战斗和结果等关键流程，以及火焰、冰霜、护盾、静态命令和动态命令的触发次数。性能监控和流程记录器互为补充：前者回答“运行是否流畅、链路是否有数据”，后者回答“功能流程是否走通、游戏行为是否被触发”。两类 CSV 共同构成论文实验章节的工程证据链。

5.9 本章小结

本章从 Unity 工程实现角度说明了《符印守卫》体感施法原型的主要模块。系统通过开始场景和战斗场景组织完整流程，通过 SpellGuardBootstrap 和 SpellGuardSceneContext 完成启动装配，通过输入系统统一 Mock、Native MediaPipe 和 ExternalBridge 三种输入源，并将手势命令接入玩家移动、法术释放、战斗流程、UI 反馈和实验记录模块。

从实现结果看，本文系统不是单独的手势识别脚本，而是一个包含输入、识别、命令、玩法、反馈、测试和数据导出的 Unity 原型。这样的工程结构能够支撑论文从方法设计、系统实现到实验验证的完整叙述，也为后续补充更复杂的手势、更多关卡或更完整的用户实验提供基础。

第 6 章 测试与实验分析

6.1 实验环境

本文实验环境如下表所示。由于最终答辩机器人和摄像头设备可能调整，表中部分硬件信息可在最终论文中根据实际测试结果补充。

项目	配置
操作系统	Windows
Unity 版本	2022.3.62f2c1

游戏引擎	Unity
输入模式	Mock / Native MediaPipe / ExternalBridge
视觉基础	MediaPipe Hands / 外部视觉桥接
测试场景	SpellGuardStart / SpellGuardPrototype
摄像头	普通 RGB 摄像头
外部桥接	UDP，本地回放或 Python 视觉程序

6.2 功能测试设计

功能测试主要验证游戏流程和手势交互是否完整。测试内容包括：开始菜单是否正常显示，菜单选项是否能够通过手势切换，确认手势是否能进入教学、训练或战斗，玩家是否能够通过手势移动，法术是否能够通过对应手势释放，敌人是否能被法术命中或控制，玩家生命和护盾是否正常变化，胜利或失败后是否能进入结算界面并重新开始。

对于三种输入模式，Mock 模式重点验证游戏逻辑和流程稳定性；Native MediaPipe 模式重点验证普通摄像头下实时识别和 Unity 内部链路；ExternalBridge 模式重点验证外部视觉结果通过 UDP 进入 Unity 后是否能被识别、显示和消费。

6.3 自动化测试

项目使用 Unity Test Framework 编写 EditMode 和 PlayMode 测试。EditMode 测试主要验证编辑器工具和数据集校验逻辑，PlayMode 测试主要验证运行时输入、识别和流程行为。

CreatePrototypeSceneTests 用于验证原型场景生成后关键对象和引用是否存在。GestureRuntimeAdapterTests 用于验证不同输入源是否能够输出统一的手势帧与命令。NativeMotionGestureRecognizerTests 和 ExternalMotionGestureRecognizerTests 用于验证动态动作识别逻辑。GestureSequenceMatcherTests 用于验证命令序列匹配是否正确处理顺序、超时和非法命令。TrainingDatasetValidatorTests 用于验证训练集文件结构是否符合要求。GesturePerformanceMonitorTests 用于验证性能采集器的字段和导出能力。

当前已归档的测试结果如表 6-2 所示。测试结果文件保存在 Docs/ExperimentResults/test_results.md 中，表中区分了已归档的重点用例和需要在最终材料中截图留存的全量测试项。

		失败		
		过数	数	验
测试范围	覆盖对象	量	量	证重点

PlayMode focused fixtures	FpsGestureMotorTests、DemoRunRecorderTests	1 3	(手 势移动 闭环、 演示流 程记 录、 CSV 导 出
PlayMode focused fixture	SpellGuardFlowControllerTests	1 令行 完成 且无 失败 输出	(结 果页统 计、战 斗重 置、流 程状态 切换
EditMode full suite	场景生成、构建配置、训练集校验	1 截图 归档	； 工 程可复 现性与 编辑器 工具正 确性
PlayMode full suite	输入运行时、动态识别、流程控制	1 截图 归档	； 运 行时交 互链路 归 档 稳定性

在已完成的 13 个 PlayMode 用例中，FpsGestureMotorTests 验证了 Point、OpenPalm、Swipe 和 BodyShift 等输入对玩家移动状态的影响，并检查禁用输入后移动状态是否清空；DemoRunRecorderTests 验证了演示流程记录器的 CSV 表头、训练与战斗转场统计、施法次数统计、导出目录安全回退和结算后自动导出机制。这些测试覆盖了从手势输入、移动响应到实验记录导出的关键链路。

自动化测试的作用是将项目从单纯演示 demo 提升为具备一定工程可靠性的原型系统。通过测试，场景生成、输入适配和识别规则的修改可以被更早发现问题，

减少答辩前临时调整带来的风险。对于尚未补充截图的全量测试，本文在局限性中如实说明其仍需通过 Unity Test Runner 进行最终截图归档。

6.4 扩展自动回放实验设计与结果

本文从渲染性能、输入吞吐、链路延迟和命令有效性四个方面评价系统性能。渲染性能包括平均 FPS、最小 FPS、平均帧耗时和 P95 帧耗时；输入吞吐包括外部 UDP 包数量和平均包间隔；链路延迟包括外部 timestamp 到 Unity 接收时间的估算延迟；命令有效性包括静态命令数量、动态命令数量和不同动态动作触发次数。

为了避免实验只停留在少量演示样例层面，本文在最小闭环实验基础上进一步构建了扩展自动回放实验。实验从本地视频归档中抽取 200 个 AVI 视频候选，使用脚本采样为图像帧目录，再通过 MediaPipe 手部关键点检测和位移规则挖掘可用动态手势片段。该过程不需要人工逐次执行动作，能够较稳定地生成训练集回放、ExternalBridge 回归和三输入模式性能统计。Jester 和 IPN Hand 等公开数据集为动态手势识别研究提供了常用的视频数据基础，也说明动态手势实验需要关注连续动作而非单帧分类[27-28]。

在更大规模的采样验证中，本文对本地 Jester 归档进行了两轮采样挖掘和外部回放验证。第一轮从 120 个目录中抽取样本，得到 84 个可检出 clip、24 个进入验证的 clip，覆盖 6 个标签；ExternalRegression 结果为 6 个模板、12 个保留验证 clip、7 个正确命中。第二轮扩大到 300 个目录后，得到 215 个可检出 clip、32 个进入验证的 clip，覆盖 8 个标签；ExternalRegression 结果为 8 个模板、16 个保留验证 clip、16 个正确命中。可以看出，随着采样规模增大、标签分布更均衡，规则模板的离线回放验证表现更稳定，但这仍属于采样挖掘和回放验证，不等同于完整监督训练或真实长期用户实验。

表 6-3 给出了 Jester 采样挖掘与回放验证的对比结果。

批次	目录数	可检出 clip	进入验证 clip	标签数	模板数	保留验证 clip	正确命中
Jester-120	120	84	24	6	6	12	7
Jester-300	300	215	32	8	8	16	16

此外，本文还对 YOLO 外部桥接路线进行了批量回放测试，以评估其作为前置视觉入口的可行性。测试从 18 个参考视频中批量运行外部桥接流程，平均 hand_ratio 为 0.1124，平均 fps 为 7.56。该结果说明，YOLO 相关链路更适合作为外部视觉前端或区域定位增强路径，而不宜直接被描述为已经完成的最终游戏判别

器；在本文工程中，真正进入 Unity 命令层的仍然是统一后的关键点序列、静态手势和动态动作事件。

扩展自动回放实验的数据规模如表 6-4 所示。200 个候选视频中有 192 个通过 MediaPipe 检出和动态位移筛选，采样帧数为 9600，检出帧数为 7359，检出比例为 0.767。经过同手标签、动作桶和 train/test 划分后，最终进入 Unity 风格模板评估的 clip 数为 48。该结果说明自动回放实验已经从最小样例扩展到较大的候选规模，但最终分类评估仍受到标签分布和模板构造规则约束。

指标	数值
候选视频数量	200
accepted mining rows	192
采样帧数	9600
MediaPipe 检出帧数	7359
检出比例	0.767
最终评估 clips	48
训练 clips	24
测试 clips	24
回放总时长	60.367 s

在性能实验中，本文对 Mock、Native MediaPipe 和 ExternalBridge 三种输入模式分别生成 9 轮回放记录，共 27 行性能数据。性能均值如表 6-5 所示。

	a	a	p9	r			stat	moti
输入模式	复 verage	verage	5_frame	in_fp	avg_p	avg_est	ic_comm	on_comm
	轮 _fps_m	_fps_s	_ms_me	s_me	acket_interv	imated_laten	ands_me	ands_mea
	数	ean	td	an	al_ms_mean	cy_ms_mean	an	n
Mock	59.501	0.320	16.927	59.079	不适用	不适用	24	13
Native MediaPipe	29.650	0.395	34.133	29.302	不适用	不适用	24	13
ExternalBridge	49.909	0.847	20.702	48.317	33.333	17.593	18	13

从表 6-5 可以看出，三种模式均能够输出统一字段的性能记录。Mock 模式不依赖摄像头和外部进程，平均 FPS 接近 60，帧耗时波动较小，适合作为流程验证和答辩兜底方案。Native MediaPipe 模式平均 FPS 约为 29.650，反映了 Unity 内部视觉链路在关键点处理和运行时更新中的额外开销。ExternalBridge 模式平均 FPS 约为 49.909，并记录了平均包间隔 33.333 ms 和估算链路延迟 17.593 ms，说明外部视觉链路不仅能够输出识别结果，也能够纳入统一性能监控。

动态手势识别结果如表 6-6 所示。该表基于外部回归验证结果统计成功、漏检和误匹配情况，并额外保留严格筛选子集作为稳定性参照。

动作类型	尝试 次数	正确 次数	漏检 次数	误匹配次 数	成 功率
body_shift_left	4	3	0	1	0.750
body_shift_right	5	4	0	1	0.800
swipe_lr	5	4	0	1	0.800
swipe_rl	2	2	0	0	1.000
strict_passed_subset	13	13	0	0	1.000

识别结果表明，规则驱动的动态识别方法能够在严格筛选子集中取得稳定结果，但在更宽松的自动挖掘样本中仍存在误匹配。误匹配主要来自不同方向和幅度动作之间的轨迹相似性，以及视频中手部运动幅度、摄像头视角和关键点检出连续性差异。由于本组数据来自离线自动回放和训练集挖掘，本文将其作为系统可复现性和工程闭环的证据，而不将其等同于长时间真实摄像头用户测试。

6.5 自定义手势增强采集实验与反馈分析

在扩展自动回放实验之外，本文还针对自定义动态手势模板进行了在线增强采集实验。该实验来自开发者在 Unity 验证界面中的真实操作，目标是观察少样本模板在加入人工确认正样本后是否能够改善识别效果。实验对象为训练集导入的动态模板。初始状态下，模板主要由 1 个原始参考样本构成，系统能够在回放自检中认回该样本，但在真实摄像头验证时出现正常动作不易触发、随机手指动作偶尔触发的问题。

增强采集过程中，系统界面记录采纳与拒绝数量，实际过程中出现十余个样本被采纳、十余个样本被拒绝的情况。拒绝原因主要包括有效帧不足、掌根位移不足、轨迹抖动比例过高和模板回放不自洽等。该过程说明，增强采集能够收集真实使用中的动作变化，但采纳样本并不一定天然适合进入正式模板库。

在第一次增强采集后，模板库中包含 1 个原始样本和 12 个增强样本。随后使用自检工具对模板内所有样本进行回放验证，结果如表 6-7 所示。

阶段	模板样本数量	自检命中数量	现象
初始模板	1	1	单一样本可被认回，但真实验证泛化不足
增强采集后	13	1	多数增强样本采纳后无法被当前规则认回
清理与规则调整后	5	5	保留样本均能自洽命中，但真实交互仍需进一步验证

第一次自检结果表明，增强采集并没有直接带来稳定提升。其根本原因在于采集质量检查和识别器匹配规则之间存在不一致：采集阶段为了收集更多正样本，放宽了置信度、位移和抖动限制；识别阶段仍存在更严格的前置运动闸门和手型噪声保护。因此，部分样本虽然通过采集质量检查，但进入模板后无法被当前识别器认回。后续调试中，系统降低了验证置信度，放宽轨迹前置闸门，并调整了手型噪声保护逻辑。

经过清理后，系统保留原始样本和 4 个增强样本，回放自检达到 5/5 命中。需要强调的是，该结果只说明模板内部样本与当前规则自洽，并不等同于真实摄像头长期使用中的高准确率。实际验证仍然存在不理想情况，说明少样本动态手势识别的难点并不只是“能否采集更多样本”，还包括样本边界定义、负样本约束、离群样本过滤和阈值自动估计等问题。

6.6 输入模式对比分析

Mock 模式的优势是稳定、可控、无需摄像头，适合自动化测试、开发调试和答辩兜底。其不足是不能证明真实视觉识别效果，因此只能作为流程和玩法验证方式。

Native MediaPipe 模式的优势是链路较短，能够展示 Unity 内部直接接入普通摄像头和 MediaPipe 手部关键点的能力。其不足是受摄像头权限、驱动、光照、背景和原生包初始化影响，需要提前在目标设备上测试。

ExternalBridge 模式的优势是算法侧灵活，便于接入 Python、离线视频、YOLO + MediaPipe 和数据回放。其不足是部署链路更复杂，并引入 UDP 通信、进程同步和时间戳对齐问题。

综合来看，Mock 模式适合演示兜底，Native MediaPipe 适合展示实时摄像头交互，ExternalBridge 适合展示外部视觉算法与 Unity 游戏系统的集成能力。

6.7 结果与局限性分析

从当前工程实现来看，系统已经完成从输入获取、手势抽象、动态识别、命令映射到 Unity 游戏反馈的完整闭环。多输入源设计提升了系统在不同环境下的可运行性，命令抽象降低了视觉层与玩法层的耦合，自动化测试和场景生成工具增强了工程可复现性。

系统仍存在一些局限。第一，当前尚未完成大规模 YOLO 手势模型训练，因此论文中不能声称系统训练了高精度 YOLO 手势分类模型；现有 YOLO 数据只适合作为外部桥接前端和区域定位增强的可行性证据。第二，当前动态手势识别主要基于规则和阈值，面对复杂动作、个体差异和长期使用场景时，可能不如深度时序模型具有适应性。第三，扩展实验虽然覆盖了 200 个候选视频、9600 个采样帧和 9 轮三模式性能记录，并额外完成了 Jester 120/300 目录的采样回放验证，但其性质仍然是离线回放和训练集自动实验，不等同于真实用户长期实验。第四，自定义手势增强采集表明，少样本模板在真实摄像头环境下容易出现正样本边界不清、离群样本污染和误触发问题，仅靠人工确认正样本不能充分解决识别鲁棒性。第五，游戏关卡、美术资源和长期内容体验仍有提升空间。

第 7 章 总结与展望

7.1 工作总结

本文围绕普通 RGB 摄像头下的体感游戏交互问题，设计并实现了一套动态手势轨迹跟踪与 Unity 游戏交互系统。系统以 MediaPipe 手部关键点和外部视觉桥接为基础，构建了 Mock、Native MediaPipe 和 ExternalBridge 三种输入源，并通过手势帧、手势命令、命令历史和动态动作事件实现输入层与玩法层解耦。

在方法层面，本文采用基于时间窗、位移阈值、状态变化和冷却机制的规则方法识别动态手势，并通过序列匹配机制为组合动作扩展提供基础。在实现层面，本文完成了《符印守卫》Unity 原型，包括开始菜单、训练、战斗、玩家移动、法术释放、敌人生成、生命护盾、HUD 反馈和性能监控等模块。在验证层面，项目通过自动化测试、功能测试设计和性能指标设计，为系统可靠性和实时性提供支撑。

7.2 主要成果

本文主要成果包括：

(1) 实现了多输入源统一的手势输入框架，使 Mock、Native MediaPipe 和 ExternalBridge 能够向上层提供一致的手势数据。

(2) 设计了 GestureFrame、GestureCommand、命令历史和序列匹配机制，使视觉识别结果能够转换为稳定、可解释、可扩展的游戏语义命令。

(3) 实现了基于动态轨迹规则的滑动、响指、指向到握拳等动作识别，并将其接入 Unity 游戏菜单、移动和施法逻辑。

(4) 完成了可运行的体感施法游戏原型《符印守卫》，形成从视觉输入到游戏反馈的闭环。

(5) 建立了场景生成、训练集校验、自动化测试和性能监控基础，为论文实验和答辩演示提供工程证据。

7.3 不足与展望

本文系统仍有进一步改进空间。首先，可以补充更大规模的数据集和用户实验，对不同光照、背景、手型、动作速度和摄像头位置下的识别稳定性进行系统评估。其次，可以引入 YOLO 前置检测，提高复杂背景下手部或人体区域定位的连续性，并与纯 MediaPipe 方案进行定量比较。再次，可以使用 LSTM、GRU 或 Transformer 等时序模型学习复杂动态手势，减少手工阈值调参成本[13-17]。最后，可以扩展双手组合手势、更多法术、更多敌人类型和完整关卡设计，使系统从毕业设计原型进一步发展为完整体感游戏作品。

参考文献

- [1] Lugaresi C, Tang J, Nash H, et al. MediaPipe: A Framework for Building Perception Pipelines[C]//Proceedings of the Third Workshop on Computer Vision for AR/VR at IEEE CVPR. 2019.
- [2] Zhang F, Bazarevsky V, Vakunov A, et al. MediaPipe Hands: On-device Real-time Hand Tracking[EB/OL]. Google AI Blog, 2019.
- [3] Google Developers. MediaPipe Solutions Guide[EB/OL]. <https://developers.google.com/mediapipe>.
- [4] Homuler. MediaPipe Unity Plugin[EB/OL]. <https://github.com/homuler/MediaPipeUnityPlugin>.
- [5] Redmon J, Divvala S, Girshick R, Farhadi A. You Only Look Once: Unified, Real-Time Object Detection[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 779-788.
- [6] Redmon J, Farhadi A. YOLO9000: Better, Faster, Stronger[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2017: 7263-7271.
- [7] Redmon J, Farhadi A. YOLOv3: An Incremental Improvement[EB/OL]. arXiv:1804.02767, 2018.
- [8] Mitra S, Acharya T. Gesture Recognition: A Survey[J]. IEEE Transactions on Systems, Man, and Cybernetics, Part C, 2007, 37(3): 311-324.

- [9] Rautaray S S, Agrawal A. Vision based hand gesture recognition for human computer interaction: a survey[J]. Artificial Intelligence Review, 2015, 43(1): 1-54.
- [10] 田秋红, 杨慧敏, 梁庆龙, 等. 视觉动态手势识别综述[J]. 浙江理工大学学报, 2020, 43(4): 557-569.
- [11] 动态手势理解与交互综述[J/OL]. 软件学报, 2021.
- [12] 高加瑞, 杜洪波, 蔡锦, 等. 基于 MediaPipe 的手势识别算法研究与应用[J]. 智能计算机与应用, 2024, 14(4): 157-161.
- [13] Molchanov P, Gupta S, Kim K, Kautz J. Hand Gesture Recognition with 3D Convolutional Neural Networks[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops. 2015.
- [14] Neverova N, Wolf C, Taylor G, Nebout F. Multi-scale Deep Learning for Gesture Detection and Localization[M]//Computer Vision - ECCV 2014 Workshops. Springer, 2014.
- [15] Tan C K, Lim K M, Chang R K Y, et al. HGR-ViT: Hand Gesture Recognition with Vision Transformer[J]. Sensors, 2023, 23(12): 5555.
- [16] Althubiti A H, Algethami H. Dynamic Gesture Recognition using a Transformer and Mediapipe[J]. International Journal of Advanced Computer Science and Applications, 2024, 15(6).
- [17] Gil-Martin M, Marini M R, Martin-Fernandez I, et al. Hand Gesture Recognition Using MediaPipe Landmarks and Deep Learning Networks[C]//Proceedings of the 17th International Conference on Agents and Artificial Intelligence. 2025: 24-30.
- [18] Sakoe H, Chiba S. Dynamic programming algorithm optimization for spoken word recognition[J]. IEEE Transactions on Acoustics, Speech, and Signal Processing, 1978, 26(1): 43-49.
- [19] Zheng Y, et al. Gesture Recognition Based on MediaPipe for Computer Game Control[J/OL]. 2023.
- [20] Olikkal P, Pei D, Karri B K, et al. Biomimetic learning of hand gestures in a humanoid robot[J]. Frontiers in Human Neuroscience, 2024, 18: 1391531.
- [21] Huda M R, Ali M L, Sadi M S. Developing a real-time hand-gesture recognition technique for wheelchair control[J]. PLOS ONE, 2025, 20(4): e0319996.
- [22] Smeddinck J D, Herrlich M, Malaka R. Exergames for Physiotherapy and Rehabilitation: A Medium-term Situated Study of Motivational Aspects and Impact on Functional Reach[C]//Proceedings of the SIGCHI Conference on Human Factors in Computing Systems. 2015.

[23] Beddiar D R, Nini B, Sabokrou M, et al. Vision-based human activity recognition: a survey[J]. Multimedia Tools and Applications, 2020, 79: 30509-30555.

[24] Multimodal Vision-based Human Activity Recognition Using Deep Learning: A Review[J/OL]. 2024.

[25] Unity Technologies. Unity Manual: Unity User Manual 2022 LTS[EB/OL]. <https://docs.unity3d.com/Manual/index.html>.

[26] Unity Technologies. Unity Test Framework Documentation[EB/OL]. <https://docs.unity3d.com/Packages/com.unity.test-framework@1.1/manual/index.html>.

[27] Materzynska J, Berger G, Bax I, Memisevic R. The Jester Dataset: A Large-Scale Video Dataset of Human Gestures[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops. 2019.

[28] Benitez-Garcia G, Olivares-Mercado J, Sanchez-Perez G, Yanai K. IPN Hand: A Video Dataset and Benchmark for Real-Time Continuous Hand Gesture Recognition[C]//International Conference on Pattern Recognition. 2020.

附录 A 工程目录说明

路径	说明
unity-spell-guard/Assets/Scripts/Core/	启动装配、流程控制、设置、场景上下文
unity-spell-guard/Assets/Scripts/Input/	手势输入、命令、动态识别和序列匹配
unity-spell-guard/Assets/Scripts/Player/	玩家移动和施法
unity-spell-guard/Assets/Scripts/Combat/	敌人、生命、护盾和战斗逻辑
unity-spell-guard/Assets/Scripts/UI/	菜单、HUD 和手势反馈
unity-spell-guard/Assets/Scripts/Diagnostics/	性能监控
unity-spell-guard/Assets/Editor/	场景生成和训练集验证
unity-spell-guard/Assets/Tests/	自动化测试

附录 B 论文素材归档计划

- [x] 补系统总体架构图。
- [x] 补输入链路图。
- [x] 补动态手势识别流程图。
- [x] 补 Unity 模块结构图。
- [x] 补扩展自动回放实验流程图。
- [x] 归档开始界面、玩法说明、实际游玩、摄像头校准、开发者实验室和自定义手势验证截图。

- ☐ 可选补充 ExternalBridge 状态、性能 CSV 导出目录和 Unity Test Runner 全量结果截图。
- ☒ 导出 Mock、Native MediaPipe、ExternalBridge 三种模式性能 CSV。
- ☐ 按学校模板调整封面、目录、页眉页脚和参考文献格式。

- 图表证据补充页

本节集中补充系统设计、动态识别、Unity 实现和实验流程中的关键图像证据，用于弥补模板版正文中图像未嵌入的问题。

相关图像均来自项目归档目录 **unity-spell-guard/Docs/ThesisAssets**，与正文第 3 章至第 6 章内容对应。

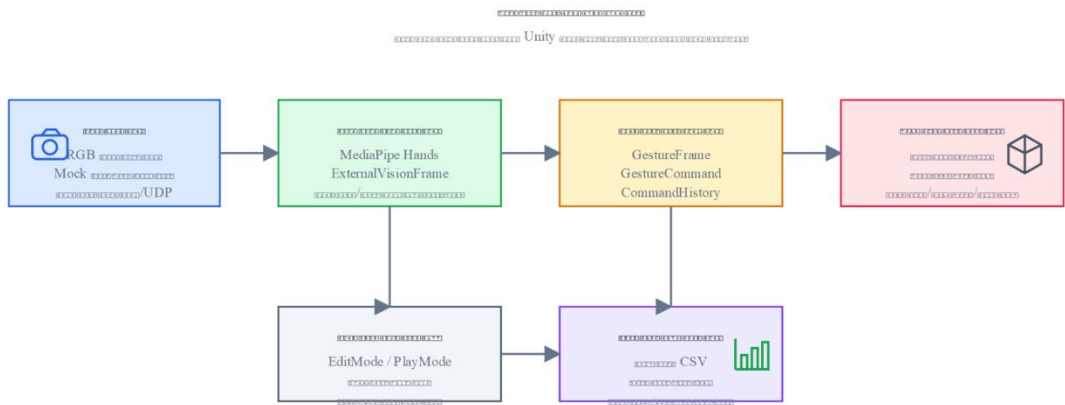


图 3-1 系统总体架构图

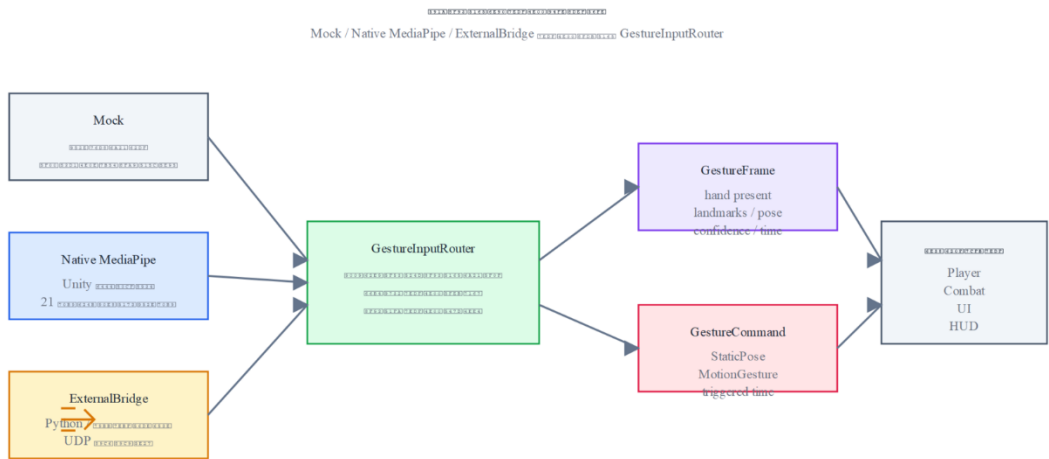


图 4-1 多输入源统一链路图

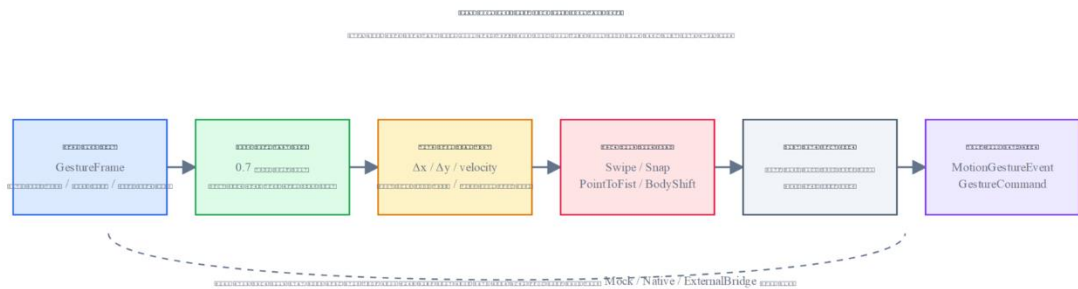


图 4-2 动态手势轨迹识别流程图

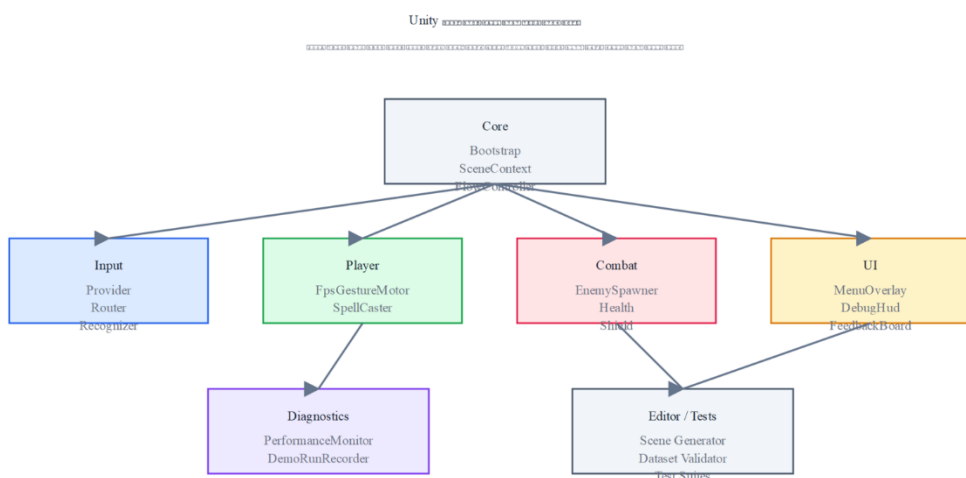


图 5-1 Unity 工程模块结构图



图 5-2 游戏开始界面



图 5-3 玩法说明界面



图 5-4 摄像头校准界面

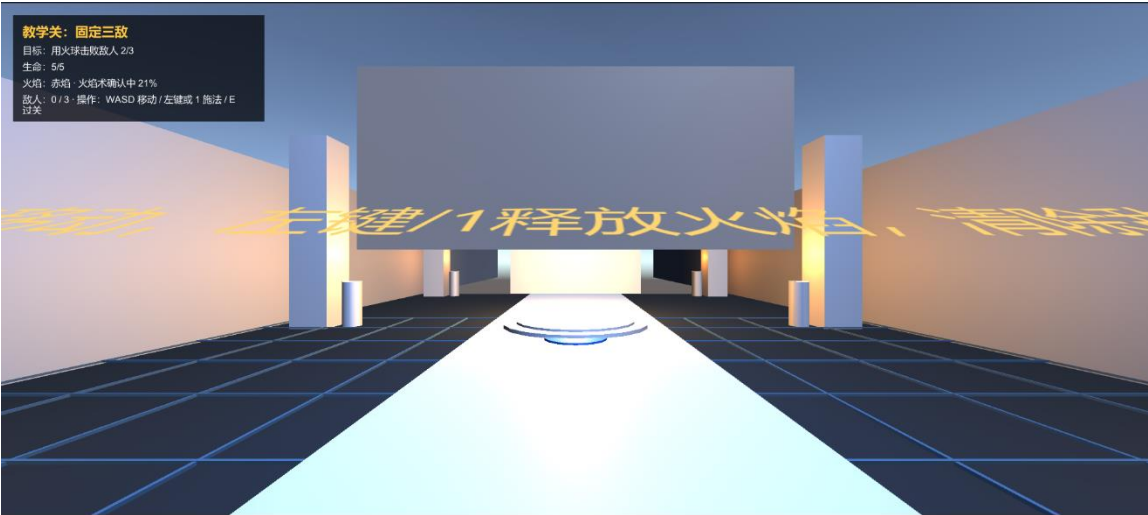


图 5-5 战斗运行界面

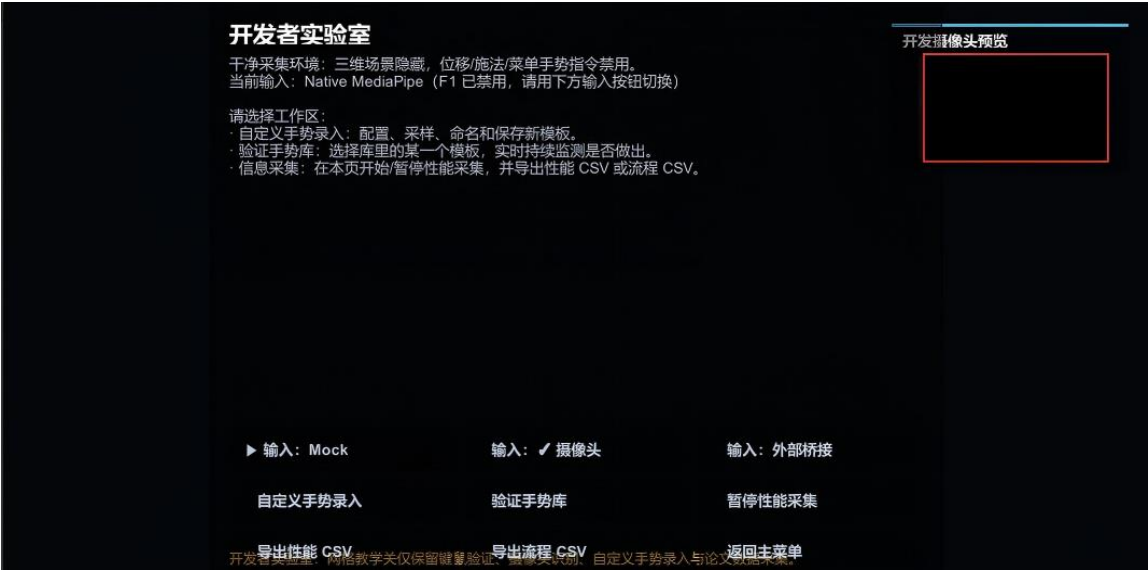


图 5-6 开发者实验室与调试入口



图 5-7 自定义动态手势验证界面

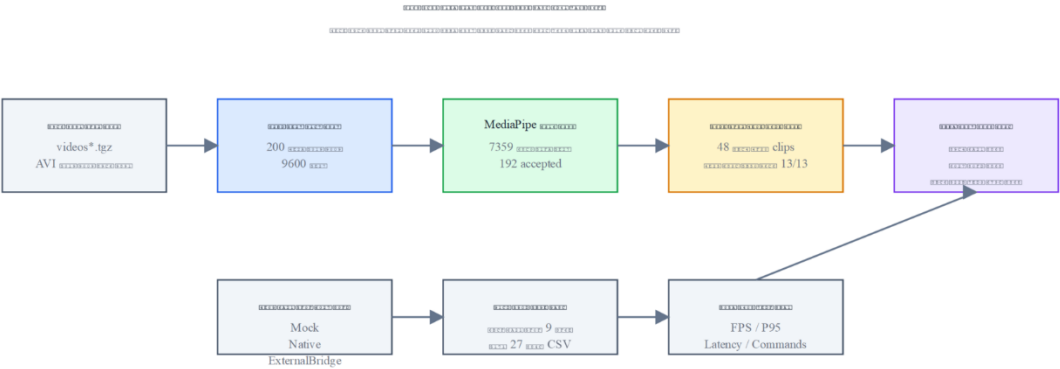


图 6-1 扩展自动回放实验流程图

表附录

表附录-1 论文实验数据与工程证据归档清单

证据类型	文件或目录	用途
系统截图	unity-spell-guard/Docs/ThesisAssets/screenshots/	支撑第 5 章系统运行效果展示
结构图	unity-spell-guard/Docs/ThesisAssets/diagrams/	支撑第 3、4、5、6 章设计与实验流程说明
性能结果	unity-spell-guard/ExperimentResults/	支撑 Mock、Native MediaPipe、ExternalBridge 三模式性能对比
回放数据	build-temp/jester_* 与 external-regression-report-*	支撑 Jester 采样挖掘与外部回放验证

致谢

大学阶段的学习和本次毕业设计的完成，离不开老师、同学和家人的帮助。首先感谢指导老师在选题、系统实现、论文撰写和材料整理过程中给予的指导，使我能够把体感游戏交互、动态手势识别和 **Unity** 工程实现逐步收束为一套可运行、可验证的毕业设计系统。

感谢计算机科学与技术学院、软件学院各位老师们在课程学习和实践训练中给予的帮助。相关课程和项目训练使我对软件工程、计算机视觉、人机交互和游戏开发有了更系统的理解，也为本课题中的输入抽象、模块划分、测试验证和论文表达提供了基础。

感谢同学和朋友在项目调试、运行截图、答辩准备和论文检查中提供的建议。毕业设计过程中遇到的许多问题并不是一次完成的，而是在不断测试、记录、修正和复盘逐渐清晰。最后感谢家人在学习生活中的支持与包容，使我能够保持稳定的节奏完成本次毕业设计。